Low-Overhead Dirty Access Tracking in GPU-UVM

Aditi Khandelia **Arush Upadhyaya** **Kushagra Srivastava** **Sankalp Mittal** **Vidhi Jain**

April 26, 2026

Project Overview

Iterative pre-copy live migration transfers memory pages from a running process to a destination host while the process continues to execute, repeating the transfer for pages that were dirtied during the previous round until the remaining working set is small enough to complete migration with a brief final pause. The efficiency of this scheme depends entirely on the ability to identify, at the end of each round, *exactly* which pages were written. For CPU memory, the Linux kernel provides this capability through soft-dirty bits embedded in page-table entries, readable via `/proc/PID/pagemap`. No analogous mechanism exists for GPU-managed memory. NVIDIA's Unified Virtual Memory (UVM) subsystem, which underpins `cudaMallocManaged` and handles demand-paging between host and device, exposes no interface for querying which pages a process has modified. Consequently, any migration or checkpointing system must conservatively copy the *entire* GPU memory footprint on every iteration, rendering iterative pre-copy migration of large GPU workloads impractical.

This project addresses that gap by implementing **per-process dirty page tracking** directly within the open-source NVIDIA UVM kernel module. UVM already intercepts every GPU page fault to manage demand-paging; we augment the write-fault servicing path to record each written page in an in-kernel data structure, without introducing any additional hardware overhead. A `procfs`-based interface allows a privileged user to start and stop tracking epochs, perform atomic cutover snapshots, pause and resume recording, and retrieve the set of pages written by a given process with **page-level granularity** and a per-page **nanosecond-resolution timestamp** suitable for ordering events across migration rounds.

The core mechanism exploits the GPU's demand-paging fault path. At the start of each tracking epoch, write permissions are *downgraded* from all currently mapped GPU pages, leaving them readable but not writable. Any subsequent GPU write generates a replayable fault; the driver's fault-servicing path records the faulting page and timestamp before restoring write permission and replaying the instruction. Pages that are only read never fault and are not recorded, eliminating the overhead of tracking read-only accesses. Formal determinism guarantees ensure that no write escapes recording and that the boundary between consecutive migration rounds is precisely defined through a barrier-protected atomic table swap.

Group Details

Following are the details of the group members:

Name	Roll No.	Email ID
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in
Sankalp Mittal	220963	sankalpm22@iitk.ac.in
Vidhi Jain	221191	vidhijain22@iitk.ac.in

Contents

Implementation

End-to-end Workflow

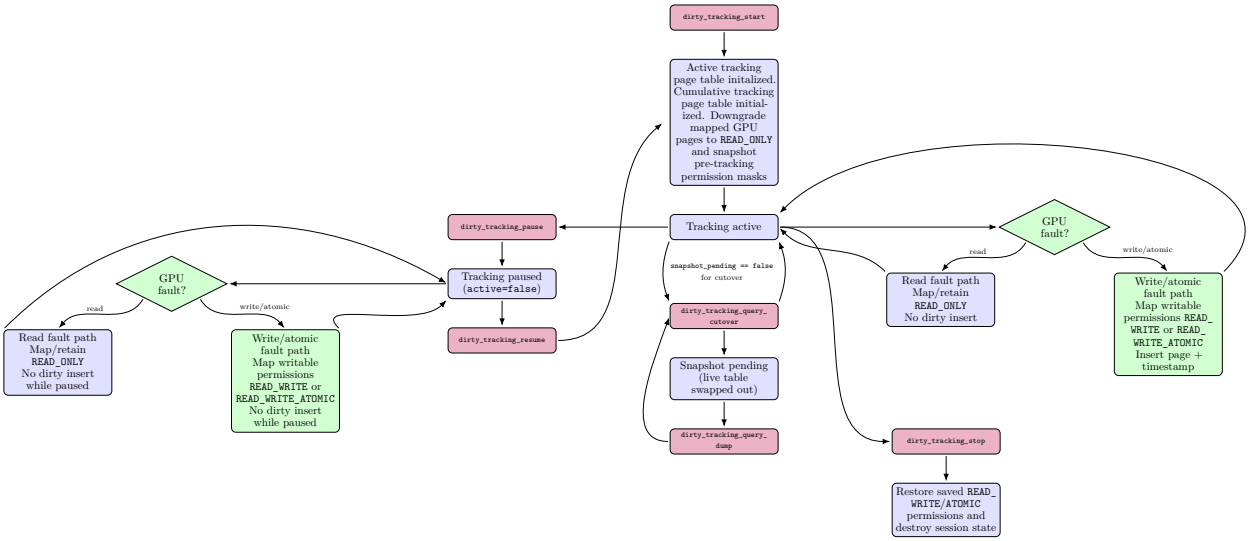

Figure 1: End-to-end workflow of the dirty page tracking system, illustrating procfs control paths, GPU fault handling, and state transitions.

State Machine

Dirty tracking is modelled as a single-owner lifecycle state machine, with all lifecycle operations serialized by `uvm_dirty_lifecycle_lock` and guarded by owner-PID checks. The states are:

1. **Tracking stopped.** `started=false`, `active=false`, `snapshot_pending=false`.
2. **Tracking active with snapshot.** `started=true`, `active=true`, `snapshot_pending=true`.
3. **Tracking active without snapshot.** `started=true`, `active=true`, `snapshot_pending=false`.
4. **Tracking paused with snapshot.** `started=true`, `active=false`, `snapshot_pending=true`.
5. **Tracking paused without snapshot.** `started=true`, `active=false`, `snapshot_pending=false`.

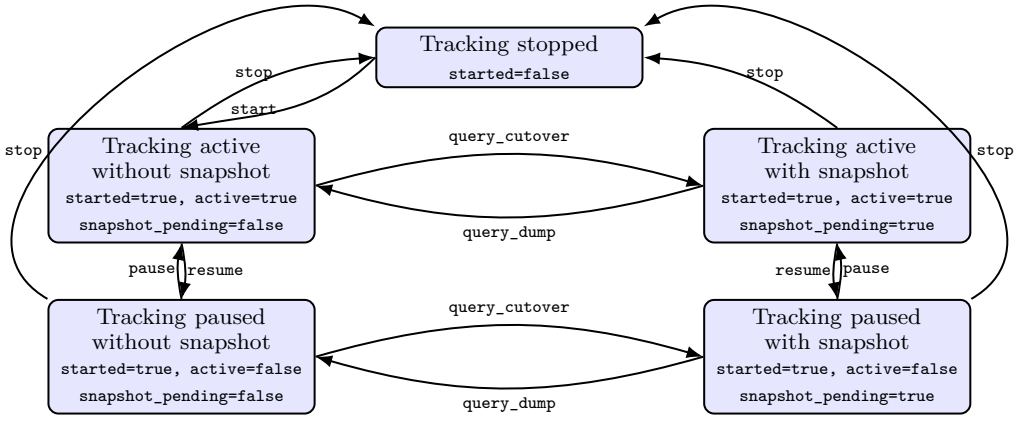

Figure 2: State-transition diagram for dirty-tracking lifecycle states.

Transitions that violate owner checks, state preconditions, or snapshot guards are rejected by the procfs handlers.

Permission Lifecycle

Figure 1 captures the permission policy used to turn hardware write access into explicit software-recorded dirty events.

Start/Resume phase At `start` (and again at `resume`), the implementation walks the owner VA space and revokes GPU write-class permissions from mapped pages, forcing tracked mappings to `READ_ONLY`. Before revocation, per-page write/atomic masks are snapshot so `stop` can restore exact pre-tracking permissions.

Active tracking phase During replayable fault servicing:

1. Read faults remain mapped as `READ_ONLY` and are not inserted into the dirty set.
2. Write/atomic faults are serviced with writable permissions (`READ_WRITE` or `READ_WRITE_ATOMIC`), and the page number plus timestamp are inserted into the live dirty table.

This enforces first-write capture while preserving normal execution after fault resolution.

Paused phase When paused, dirty recording is disabled, but fault handling still progresses: read paths remain `READ_ONLY`, while write/atomic paths may regain writable permissions for correctness and forward progress. Because `active=false`, no dirty inserts are emitted in this interval.

Cutover/Dump phase `query_cutover` requires `snapshot_pending==false`. Under the query barrier, it swaps out the live table into a frozen snapshot and installs a new live table so recording can continue. `query_dump` then consumes that frozen snapshot: in `delta` mode it emits snapshot contents directly; in `cumulative` mode it merges snapshot entries into the cumulative table before output.

Stop phase `stop` restores the previously captured per-page write/atomic permission masks and tears down live, snapshot, and cumulative tracking state, returning the system to its pre-tracking permission configuration.

Modified Driver Files

All driver modifications reside under `kernel-open/nvidia-uvm/`. The files containing functionality added or significantly modified by us are:

1. **`uvm_common.h` / `uvm_common.c`**
   The primary implementation file. Defines `struct dirty_page_info` (page number, timestamp) and the three global dirty-set tables (`g_dirty_ds`, `g_dirty_ds_snapshot`, `g_dirty_ds_cumulative`). Implements table lifecycle (`uvm_dirty_page_table_init`, `uvm_dirty_page_table_destroy`), fault recording (`uvm_dirty_page_table_record`), snapshot management (`uvm_dirty_new_snapshot`, `uvm_dirty_merge_snapshot_into_cumulative_locked`), user hints to select the data structure backend (`dirty_tracking_hint_write`), and all nine `procfs` handler implementations including `uvm_dirty_procfs_init`. Exports function pointers (`uvm_dirty_invalidate_fn`, `uvm_dirty_activate_now_fn`, `uvm_dirty_barrier_end_fn`, `uvm_dirty_query_barrier_begin_fn`, `uvm_dirty_query_barrier_end_fn`) that are registered from `uvm_va_space.c` at module init.

2. **`uvm_dirty_ds.h`**
   Defines the `uvm_dirty_ds_ops` interface (init, insert, lookup, for_each_in_range, destroy), the `uvm_dirty_ds` struct holding ops pointer, backend-private data, and statistics, and inline timing wrappers for all operations.

3. **`uvm_dirty_ds_xarray.c`**
   xarray backend using atomic `xa_cmpxchg` for first-write-wins insert.

4. **`uvm_dirty_ds_vector.c`**
   Sorted dynamic array backend with binary-search insert and lookup.

5. `uvm_dirty_ds_vector_unsorted.c`
   Unsorted append-order vector with spinlock, preserving fault-arrival order.

6. `uvm_dirty_ds_linked_list.c`
   Singly-linked list with sentinel node.

7. `uvm_dirty_ds_nested_bitmap.c`
   Two-level hierarchical bitmap for sparse page sets.

8. `uvm_dirty_ds_chunked.c`
   Chunked vector with a background kthread sorter for deferred ordering.

9. `uvm_gpu_replayable_faults.c`
   The write-fault hook. After standard fault servicing, checks `uvm_dirty_tracking_active()` and `service_access_type >= UVM_FAULT_ACCESS_TYPE_WRITE`, then calls `uvm_dirty_page_table_record` with the faulting page number and `ktime_get_ns()` timestamp.

10. `uvm_va_space.c`
    Implements the barrier infrastructure. `uvm_dirty_downgrade_all_permissions` walks all va-ranges and va-blocks for the owner's va_space, calling `uvm_va_block_revoke_prot_mask` with `UVM_PROT_READ_WRITE` to strip write access from all GPU PTEs, followed by `uvm_tracker_wait` to confirm GPU processing. `uvm_dirty_query_barrier_begin` and `uvm_dirty_query_barrier_end` implement the cutover barrier by locking all parent GPU replayable-fault ISRs and spinning until the fault buffer is empty. `uvm_va_space_dirty_init` registers all five function pointers at module init.

11. `uvm_va_space.h`
    Declares `uvm_va_space_dirty_init`, the registration hook invoked during module initialisation.

12. `uvm_va_block.c`
    `compute_new_permission` modified to force `READ_ONLY` on read faults when tracking is active for the faulting va_space's owner, preventing read faults from obtaining write mappings that would bypass recording.

13. `uvm_migrate.c`
    `block_migrate_map_unmapped_pages` modified to map GPU-migrated pages as `READ_ONLY` when tracking is active, closing the `cudaMemPrefetchAsync` bypass.

14. `uvm_va_block.h`
    Extends per-GPU va-block state with `dirty_downgraded_pages` and `dirty_downgraded_atomic_pages` masks used to restore exact pre-tracking permissions at stop.

15. `uvm_procfs.c`
    Integrates the dirty-tracking `procfs` interface into `/proc/driver/nvidia-uvm/` by invoking `uvm_dirty_procfs_init(uvm_proc_dir)` during procfs setup.

16. `nvidia-uvm-sources.Kbuild`
    Adds all dirty data-structure backend source files (`uvm_dirty_ds_xarray.c`, `uvm_dirty_ds_bitmap.c`, `uvm_dirty_ds_vector.c`, `uvm_dirty_ds_vector_unsorted.c`, `uvm_dirty_ds_chunked.c`, `uvm_dirty_ds_nested_bitmap.c`, `uvm_dirty_ds_linked_list.c`) to the module build list.

17. `uvm.c`
    Top-level integration; calls `uvm_va_space_dirty_init` during module initialisation so callback pointers are registered before tracking control paths execute.

Procfs Interface

Nine procfs entries are created under `/proc/driver/nvidia-uvm/`:

1. **dirty_tracking_start (write)**
   Input: "<pid> <delta|cumulative>"
   Initializes a fresh live table for the target PID, selects query mode (delta or cumulative), executes permission downgrade over the owner's managed GPU mappings, and finally enables recording.
   The request is rejected if tracking is already started, if the input format is invalid, or if initialization/callback setup fails.

2. **dirty_tracking_stop (write)**
   Input: "<pid>"
   Restores previously downgraded GPU page permissions, destroys all tracking state (live/snapshot/cumulative), clears snapshot-pending state, and flushes accumulated DS statistics to /tmp/uvm_dirty_ds_stats.txt.
   The request is rejected if the supplied PID is not the active owner.

3. **dirty_tracking_pause (write)**
   Input: "<pid>"
   Establishes the query barrier, sets active=false, and then releases the barrier. This ensures pause is synchronized with replayable-fault ISR state.
   The request is rejected if the PID does not match the active owner or if tracking is not currently active.

4. **dirty_tracking_resume (write)**
   Input: "<pid>"
   Re-executes the same downgrade/activate/barrier-end sequence used by start, re-establishing deterministic write-fault capture after a paused interval.

5. **dirty_tracking_query_cutover (write)**
   Input: "<pid>"
   Acquires the query barrier, atomically swaps live $\rightarrow$ snapshot under write lock, installs a fresh live table, updates epoch metadata, sets g_snapshot_pending=true, and releases the barrier.
   The request is rejected if the previous snapshot has not yet been consumed by query_dump.

6. **dirty_tracking_query_dump (read)**
   No input payload (read endpoint). Requires g_snapshot_pending=true.
   In delta mode, emits only the current frozen snapshot. In cumulative mode, first merges snapshot contents into g_dirty_ds_cumulative (first-write-wins), then emits the cumulative union.
   Output includes a header line followed by one line per page: 0x*addr timestamp_ns*.
   Snapshot consumption is overflow-safe: when seq_file overflows, snapshot state is retained and replayed on the next read.

7. **dirty_ds_stats_toggle (write)**
   Input: "enable" or "disable"
   Enables or disables instrumentation for DS and callback timing counters. Writing enable also resets all counters/timers to zero to provide a clean measurement window.

8. **dirty_ds_stats (read)**
   Read-only statistics snapshot for the current session.
   Reports operation counts and average latency for init/insert/lookup/for-each/destroy, average lock-wait components, and callback timing for invalidate, activate-now, and barrier-end.

9. **dirty_tracking_hint (write)**
   Input: "WRITE_SEQ" or "WRITE_RAND"
   Switches the data structure backend between uvm_dirty_ds_vector when the input is WRITE_SEQ and uvm_dirty_ds_nested_bitmap when the input is WRITE_RAND. Since the vector gives better performance for sequential writes and the nested bitmap gives better performance for random writes, this allows users to select the best backend for their workload.

Design Choices

Determinism Guarantees

Write-Capture Determinism The implementation provides deterministic *first-write capture* for GPU pages while tracking is active. A page is recorded exactly when its first qualifying write/atomic fault is serviced after permissions have been re-armed to read-only.

1. **Arming at lifecycle boundaries.** `dirty_tracking_start` and `dirty_tracking_resume` invoke the registered downgrade callback, which walks the owner VA-space and revokes GPU write-class permissions (`READ_WRITE/ATOMIC`) from currently mapped pages. This forces subsequent writes to fault before regaining writable mappings.
2. **No non-write false positives, no prefetch bypass.** Dirty insertion is gated to write-class accesses only. Read/prefetch paths are kept read-only for the tracked owner, and GPU migration of previously unmapped pages under active tracking is also mapped read-only, so `cudaMemPrefetchAsync`-style migrations do not silently bypass write-fault capture.
3. **Single insertion point in fault service.** Recording occurs in replayable fault servicing only when `active=true` and `service_access_type >= UVM_FAULT_ACCESS_TYPE_WRITE`, at which point the page number and `ktime_get_ns()` timestamp are inserted into the live table.
4. **Exactly-once entry semantics per live table (epoch).** Backend insertion is first-write-wins (duplicates are ignored), so within a given live table, repeated writes to the same page do not create duplicate entries or overwrite the original timestamp. In `delta` mode, each `cutover` installs a fresh live table, so the same page may appear again in a later epoch only if it faults again after mappings become faultable.

This yields deterministic capture semantics for migration accounting: each qualifying write/atomic fault contributes one stable dirty record in the current live table. Subsequent writes to an already writable page are not re-faulted, so they are not re-recorded until mappings become faultable again (for example at `resume` or a new `start`).

Epoch Determinism on `start/resume` The epoch entry path is implemented through the same callback chain in both `start` and `resume`: `uvm_dirty_invalidate_fn()` → `uvm_dirty_activate_now_fn()` → `uvm_dirty_barrier_end_fn()`.
In `uvm_va_space_dirty_init`, these are registered to `uvm_dirty_downgrade_all_permissions`, `uvm_dirty_activate_now`, and `uvm_dirty_barrier_end` respectively.

1. **Begin-side locking and ownership resolution.** `uvm_dirty_downgrade_all_permissions` calls `uvm_dirty_barrier_begin`, which acquires `g_uvm_global.va_spaces.lock` and resolves the unique owner `va_space` for the tracking PID, returning an error if there is not exactly one `va_space` registered under that TGID and keeping `va_spaces.lock` held on success. Unlike the query barrier (used in pause/cutover), the start barrier does *not* spin on the fault buffer; it only acquires locks. This is safe because the immediately following `va_space_down_write` lock prevents all fault service during the critical section, so no faults need to be flushed. `uvm_dirty_downgrade_all_permissions` then calls `uvm_va_space_down_write(va_space)` itself, bringing the VA-space write lock under the same critical section as `va_spaces.lock`.
2. **Permission downgrade while barrier is held.** Under both locks, it walks all managed VA ranges and VA blocks, snapshots per-page write/atomic PTE bitmasks into `dirty_downgraded_pages/dirty_downgraded_atomic_pages` for later restoration at `stop`, revokes GPU write-class permissions via `uvm_va_block_revoke_prot_mask`, and drains each block's GPU tracker (`uvm_tracker_wait`) to confirm GPU TLB invalidations have completed before advancing to the next block.
3. **Activate-before-end ordering.** On success, downgrade returns with both barrier locks still held (`va_spaces.lock` and the `va_space` write lock). The handler then calls `uvm_dirty_activate_now_fn` (`active=true`) while both locks are still held, and only afterwards calls `uvm_dirty_barrier_end_fn`, which releases `uvm_va_space_down_write` and `g_uvm_global.va_spaces.lock`. This ordering eliminates two race windows that would silently lose first-write events:
   - **Race A - active before downgrade.** If `active=true` were set before pages are revoked to read-only, writes to still-writable pages would complete without faulting and would not be recorded.
   - **Race B - downgraded but not yet active.** If pages were revoked to read-only while `active=false`, write faults arriving in that window would be serviced (the fault handler grants `READ_WRITE` for forward progress) but not inserted into the dirty table, silently missing the first-write event.

   Holding `uvm_va_space_down_write` throughout the transition prevents all fault service for the owner va-space during the critical section. After the function returns and locks are released, any write fault to a

downgraded page is recorded deterministically because both conditions are satisfied: `active=true` and permissions are `READ_ONLY`.

4. **`start` semantics.** `start` first creates fresh table state (and cumulative state in cumulative mode), then executes the activation chain, so it establishes a new epoch from a clean live table.
5. **`resume` semantics.** `resume` is accepted only for owner PID with `started=true, active=false`. It re-runs the same lock+downgrade activation chain without reinitializing tables, so recorded entries are preserved across pause/resume.
6. **Precise inclusion/exclusion boundary.** Writes before successful `start` are outside the epoch; writes during pause are excluded (`active=false`); writes after successful `start`/`resume` are included through the write-fault capture path.

Epoch Determinism on `pause` The pause operation must establish a precise cutoff where all faults in-flight at the moment of pause are resolved before recording is disabled. This ensures the dirty set captures a deterministic boundary: all writes before pause are recorded; all writes after pause are not.

1. **Query barrier for synchronization.** `dirty_tracking_pause` acquires the query barrier (`uvm_dirty_query_barrier_begin`), which locks all GPU replayable-fault ISRs and spins until the fault buffer is empty. This ensures no fault is mid-service when recording is disabled.
2. **Atomic recording disable.** While the query barrier is held (ISRs locked, fault buffer empty), `active` is set to `false` via `WRITE_ONCE` to prevent the compiler from optimizing or reordering the write. Subsequent faults arriving at the fault handler see `active=false` and skip dirty insertion.
3. **Barrier release enables new faults.** After setting `active=false`, the query barrier is released, unblocking ISRs and fault buffer processing. Any write faults after this point are serviced (mappings remain faultable) but not recorded, ensuring the pause boundary is sharp and deterministic.
4. **Preserved state across pause/resume.** Pause does not modify permissions or destroy tables. The live table, permission state, and accumulated entries remain intact. Resume re-executes the downgrade+activate sequence without reinitializing, so the epoch continues deterministically.

Epoch Determinism on `stop` The stop operation must guarantee that the tracking session is completely torn down and permissions are exactly restored to their pre-tracking state. This ensures deterministic cleanup and prevents resource leaks or stale permissions.

1. **Permission restoration from snapshot.** At `stop`, `uvm_dirty_restore_fn` is called, which walks all VA blocks for the owner and restores the per-page write/atomic permission masks that were captured at `start`/`resume`. This restores GPU PTEs to their exact pre-tracking configuration, undoing the downgrade and enabling future non-tracked access.
2. **Table destruction and state cleanup.** `uvm_dirty_page_table_destroy(false)` deallocates all three tables (live, snapshot, cumulative), freeing associated backend resources. The global state is reset: `started=false`, `active=false`, and `snapshot_pending=false`. Statistics are flushed to `/tmp/uvm_dirty_ds_stats.txt` for post-session analysis.
3. **Ownership and state validation.** `stop` validates that the requesting PID matches the current owner (if tracking was started for a specific PID). This prevents unauthorized cleanup and ensures teardown is tied to the correct session.
4. **Deterministic session boundary.** The combination of permission restoration, table destruction, and state reset establishes a precise end-of-epoch boundary. Any query cutover/dump operations must complete before `stop`; once `stop` succeeds, the session is completely dismantled and a new session can be initialized.

Epoch Determinism on `query_cutover` The cutover operation must establish a precise snapshot boundary such that the dirty set can be consistently queried without blocking new fault recording. All faults before cutover are in the old live table; all faults after belong to the new live table.

1. **Snapshot guard and query barrier.** `query_cutover` first validates that no previous snapshot is pending (`g_snapshot_pending==false`), ensuring only one snapshot exists at a time. It then acquires the query barrier (`uvm_dirty_query_barrier_begin`), which locks all GPU replayable-fault ISRs and spins until the fault buffer is empty.
2. **Atomic table swap and epoch increment.** While the query barrier is held and the DS write lock is acquired, the operation performs three atomic steps: (1) saves current live table to snapshot (`g_dirty_ds_snapshot = g_dirty_ds`), (2) installs fresh live table (`g_dirty_ds = new_live`), and (3) increments the epoch counter (`g_dirty_epoch += 1`). The snapshot epoch is synchronized (`g_dirty_snapshot_epoch = g_dirty_epoch`).

3. **Precise cutover boundary.** Because the query barrier holds ISRs locked and the fault buffer is confirmed empty during the swap, no fault service can interleave with the table swap. Faults arriving after barrier release see the new live table; those in-flight during swap are guaranteed to have completed before the swap (or arrived after and went to new table).
4. **Snapshot consumption guard.** After cutover, `g_snapshot_pending` is set to `true`, preventing a new cutover until the snapshot is consumed by `query_dump`. This enforces strict cutover-dump ordering and prevents snapshot loss.

Epoch Determinism on `query_dump` The dump operation must return a consistent snapshot of the dirty set at the time of the preceding cutover. The output must reflect exactly the pages recorded between the previous dump and the current cutover.

1. **Snapshot presence and mode validation.** `query_dump` first validates that a snapshot exists and is pending (`g_snapshot_pending==true` and `g_dirty_ds_snapshot.priv != NULL`). The query mode (delta or cumulative) is checked to determine which table to emit.
2. **Cumulative merge with first-write-wins.** In cumulative mode, the snapshot is merged into the cumulative table (`uvm_dirty_merge_snapshot_into_cumulative_locked`) using first-write-wins semantics: entries already in cumulative are not overwritten. This preserves the earliest timestamp for each page across all query windows in the session.
3. **Consistent read under lock.** The read lock on `uvm_dirty_ds_rwsem` is held while iterating the query table (`uvm_dirty_ds_for_each_in_range`). This prevents the table from being modified (cutover swaps require write lock) and ensures the output is consistent with the table state at read-lock acquisition.
4. **Overflow-safe snapshot persistence.** If the seq_file buffer overflows (`seq_has_overflowed`), the snapshot is preserved (not destroyed) and `g_snapshot_pending` remains true. The caller retries the read with a larger buffer, and the snapshot is replayed identically. Only when the entire output fits is the snapshot destroyed and `g_snapshot_pending` cleared, allowing the next cutover.

Query Modes

The tracking system supports two query modes, selected at `dirty_tracking_start` time and fixed for the lifetime of the session. The mode governs how `query_dump` assembles its output from the three-table pipeline (`g_dirty_ds`, `g_dirty_ds_snapshot`, `g_dirty_ds_cumulative`).

Delta mode In delta mode the output of each `query_dump` reflects only the pages written between the immediately preceding `query_cutover` and the one before it, a per-epoch incremental diff.

1. **Epoch scoping.** `query_cutover` atomically promotes the live table to the snapshot and installs a fresh empty live table. All write faults that arrived before the barrier are captured in the frozen snapshot; all faults after the barrier go to the new live table. Because the live table starts empty on every cutover, the snapshot contains exactly and only the pages first-written in the elapsed epoch.
2. **Dump behaviour.** `query_dump` reads the snapshot directly without touching `g_dirty_ds_cumulative`. It emits one line per entry (`0x`*`addr timestamp_ns`*), then destroys the snapshot and clears `g_snapshot_pending`, allowing the next cutover.
3. **Use case.** Delta mode is suited to iterative pre-copy migration: each round transfers the pages reported in the previous epoch's dump, so the working set of remaining dirty pages shrinks toward zero over successive rounds. Because the cumulative table is never populated, delta mode also has lower memory overhead during long sessions with many cutovers.

Cumulative mode In cumulative mode each dump emits the union of all pages written since `dirty_tracking_start`, regardless of how many cutover/dump cycles have elapsed.

1. **Snapshot merge.** Before emitting output, `query_dump` calls `uvm_dirty_merge_snapshot_into_cumulative_locked`, which iterates every entry in the frozen snapshot and inserts it into `g_dirty_ds_cumulative` using first-write-wins semantics: if the page is already present in the cumulative table its original timestamp is preserved; if absent, the snapshot's timestamp is adopted. After the merge the snapshot is destroyed and `g_snapshot_pending` is cleared.
2. **Dump behaviour.** `query_dump` then iterates `g_dirty_ds_cumulative` (not the now-destroyed snapshot) and emits its full contents. The output therefore grows monotonically: a page that appeared in any earlier epoch remains visible in every subsequent dump, with its earliest recorded timestamp.

3. **First-write-wins across epochs.** Because the merge step is first-write-wins and the cumulative table is never cleared during an active session, the timestamp associated with each page address always reflects the first qualifying write fault observed since `start`, even if the same page faulted again in a later epoch. This provides a stable, session-global dirty set for workloads where the caller needs to reason about *total* footprint rather than incremental change.
4. **Use case.** Cumulative mode is suited to final-copy migration: after all iterative rounds complete, a single cumulative dump identifies every page touched during the entire session, providing the complete set that must be transferred before the workload is suspended.

Mode interaction with `pause/resume` Both modes are unaffected by pause/resume cycles at the table level. During a paused interval `active=false`, so no inserts reach the live table; the cumulative table is not modified either. On `resume`, the live table continues accumulating into whichever epoch was current at pause time; a subsequent `query_cutover` captures exactly the pages written before pause plus those written after resume, with the paused interval contributing nothing to the dirty set. This keeps both delta and cumulative semantics well-defined across pause/resume boundaries.

Data Structures

Global state Three `uvm_dirty_ds` instances hold the tracking data:

1. `g_dirty_ds` live table; the fault handler inserts here on every qualifying write fault while `active=true`.
2. `g_dirty_ds_snapshot` frozen copy created by `query_cutover`; exists only between a cutover and the subsequent `query_dump`. Once consumed the pointer is set to `NULL`.
3. `g_dirty_ds_cumulative` session-union table used in cumulative mode; each `query_dump` merges the snapshot into this table with first-write-wins semantics before emitting output.

Backend abstraction layer The three tables all share a single generic interface defined in `uvm_dirty_ds.h`. The design follows the ops-table (vtable) pattern common in the Linux kernel: a `struct uvm_dirty_ds_ops` holds five function pointers, and each backend fills in its own implementations at module link time. This separates the *interface* (what operations exist and what they mean) from the *implementation* (how a particular data structure carries them out), so a new backend can be plugged in by pointing `g_dirty_ds.ops` at a different ops struct no other code needs to change.

The ops table (`struct uvm_dirty_ds_ops`)

- **`init(ds)`** allocates and initialises the backend-private state, storing the pointer in `ds->priv`. Called once per table lifetime under the DS write lock.
- **`insert(ds, page_num, timestamp)`** records a dirty page with its first-write timestamp. All backends implement first-write-wins: if `page_num` is already present the call is a no-op and the original timestamp is preserved. Called from the fault-handler hot path.
- **`lookup(ds, page_num)`** returns a pointer to the `dirty_page_info` for the given page, or `NULL` if not dirty. Used by the cumulative merge path and the snapshot-persistence guard.
- **`for_each_in_range(ds, start, end, fn, ctx)`** iterates all dirty pages whose page number falls in [*start*, *end*] and invokes the callback `fn(page_num, timestamp, ctx)` for each. Used by `query_dump` to emit output and by the cumulative merge path.
- **`destroy(ds)`** frees all entries and the backend-private state, then sets `ds->priv = NULL`. Called under the DS write lock at `stop` or on error unwind.

The container (`struct uvm_dirty_ds`) Each of the three global tables is an instance of:

```
struct uvm_dirty_ds {
    const struct uvm_dirty_ds_ops *ops;
    void                          *priv;
    struct uvm_dirty_ds_stats      stats;
};
```

`ops` is a compile-time constant pointing to the chosen backend. `priv` is the backend-private allocation (e.g. the `struct xarray*` for the xarray backend); it is `NULL` when the table is not initialised, which is how `uvm_dirty_tracking_started()` checks whether a session is active. `stats` holds per-table instrumentation counters (see below).

Inline timing wrappers Rather than scattering timing code into each backend, `uvm_dirty_ds.h` provides five `static inline` wrappers one per op that intercept every call when `ds->stats.enabled` is true:

1. Record `ktime_get()` before the call.
2. Dispatch through `ds->ops->op(...)`.
3. Add `ktime_to_ns(ktime_sub(ktime_get(), t0))` to the corresponding `atomic64_t` accumulator.
4. Increment the matching `atomic_long_t` count.

The `unlikely()` guard on `stats.enabled` means the check is predicted-not-taken in normal operation, so the instrumentation path imposes no measurable overhead when disabled. Because both the accumulator and the count are atomics, concurrent fault-handler threads can update them without a lock.

Instrumentation via procfs The two procfs entries `dirty_ds_stats_toggle` and `dirty_ds_stats` expose the accumulated measurements:

- Writing `"enable"` to `dirty_ds_stats_toggle` sets `stats.enabled = true` *and* resets all counters to zero, giving a clean measurement window. Writing `"disable"` stops accumulation.
- Reading `dirty_ds_stats` calls `nv_procfs_read_dirty_ds_stats`, which reports for each operation: call count, average end-to-end latency in nanoseconds, and average time spent blocked waiting for the DS rwsem. It also reports average latency for the three lifecycle callbacks (`invalidate`, `activate_now`, `barrier_end`). On `stop`, the same counters are flushed to `/tmp/uvm_dirty_ds_stats.txt` for offline analysis.

The lock-wait fields (`insert_lock_wait_ns`, `lookup_lock_wait_ns`, `for_each_lock_wait_ns`) measure the time each hot-path call spends blocked in `down_read` before reaching the backend, separating contention cost from pure algorithmic cost. This lets us attribute latency to lock pressure vs. backend complexity when comparing backends under load.

Backend implementations Seven backends are implemented, each compiled unconditionally into the module. The active backend is selected at compile time by setting the `.ops` pointer in the static `g_dirty_ds` initialiser; switching requires only a one-line source change. Table 1 summarises the key properties of each.

Backend	Insert	Lookup	Iterate	Notes
bitmap	$O(1)$	$O(1)$	$O(P/w + k)$	flat `vzalloc` bitmap; **no timestamps**; 32 MB fixed
xarray	$O(1)$ amortised	$O(1)$	$O(n)$	atomic `xa_cmpxchg`; timestamps stored
sorted vector	$O(n)$	$O(\log n)$	$O(\log n + k)$	binary-search insert; no internal lock
unsorted vector	$O(n)$	$O(n)$	$O(n)$	append-only; IRQ-safe spinlock
linked list	$O(n)$	$O(n)$	$O(n)$	sentinel head; no extra lock
nested bitmap	$O(1)$	$O(1)$	$O(B + k)$	2-level xarray+bitmap; timestamps stored
chunked vector	$O(1)$ amortised	$O(\log n + n_c)$	$O(n)$	background kthread sort

Table 1: Backend summary. n = dirty page count, k = pages in query range, P = `DIRTY_BITMAP_MAX_PAGES` (2^{38}), w = word size in bits, B = number of allocated VA blocks, n_c = entries in active (unsorted) chunk.

Bitmap (`uvm_dirty_ds_bitmap.c`) A flat `vzalloc`'d bitmap of 2^{38} bits, covering up to 246 TB VA with a fixed 32 MB allocation. Insert is a single atomic `set_bit`: $O(1)$, no per-entry allocation. Lookup is `test_bit`: $O(1)$, returning a static sentinel rather than a heap-allocated `dirty_page_info` since timestamps are not stored. Iteration walks the bitmap with `find_next_bit`: $O(P/w + k)$. The fixed allocation dominates memory cost for sparse working sets; in return, the insert path is the cheapest of all backends. Timestamps are silently discarded, so this backend is unsuitable when per-page first-write ordering is required.

xarray (`uvm_dirty_ds_xarray.c`) Uses the Linux `xarray` (a radix-tree variant) keyed by page number. Insert performs a fast non-atomic `xa_load` check first; if the page is absent it allocates a `dirty_page_info` and calls `xa_cmpxchg(GFP_ATOMIC)`, which atomically inserts only if the slot is still empty. A non-`NULL` return from `xa_cmpxchg` means another thread won the race (first-write-wins), so the redundant allocation is freed. Lookup is a single `xa_load`. Iteration uses `xa_find` with an advancing index. Memory is proportional to the number of dirty pages; the xarray allocates nodes on demand.

Sorted vector (`uvm_dirty_ds_vector.c`) Maintains a `kvmalloc`'d pointer array kept sorted in ascending page number order at all times. Insert uses binary search (`vec_lower_bound`) to locate the insertion point in $O(\log n)$, then `memmove` to shift the tail in $O(n)$. Deduplication is a pointer comparison at the found position. Lookup is binary search: $O(\log n)$. `for_each_in_range` binary-searches to the start of the range and then walks sequentially. The array doubles in capacity when full (`kvmalloc_array + memcpy + kvfree`). This backend relies on the outer DS rwsem for thread safety and carries no internal lock (the commented-out spinlock was removed in favour of the rwsem).

Unsorted vector (`uvm_dirty_ds_vector_unsorted.c`) A variant that appends entries in fault-arrival order without sorting. Insert performs an $O(n)$ linear dedup scan then an $O(1)$ tail append. Lookup and `for_each_in_range` are both $O(n)$ linear scans. The backend embeds its own `spinlock_t` and uses `spin_lock_irqsave` / `spin_unlock_irqrestore` around every mutation, making it safe in IRQ context independently of the outer rwsem. This backend trades query speed for insertion simplicity and fault-arrival-order preservation.

Linked list (`uvm_dirty_ds_linked_list.c`) A singly-linked list with a sentinel head node whose `info` field is `NULL`. Both the list struct and each `dirty_ll_node` are heap-allocated. Insert walks the list for dedup ($O(n)$) then appends a new tail node. Lookup and iteration are also $O(n)$ walks. The list carries no internal lock, relying on the outer rwsem. It serves primarily as a correctness baseline and debugging aid.

Nested bitmap (`uvm_dirty_ds_nested_bitmap.c`) A two-level structure: an `xarray` maps VA-block indices (each block covers 512 pages, matching the UVM 2 MB VA-block size) to `struct block_entry` values, each of which holds a 512-bit bitmap and a 512-element timestamp array. Insert derives the block index and in-block offset in $O(1)$, calls `xa_cmpxchg` to lazily allocate the block entry on first use, then issues `test_and_set_bit`: if the bit was clear (first write to this page) it stores the timestamp with `WRITE_ONCE`; if already set it is a no-op. Lookup is `xa_load` + `test_bit`: $O(1)$. Iteration walks only the allocated blocks via `xa_find`, then uses `find_next_bit` within each block's bitmap, giving $O(B + k)$ where B is the number of allocated blocks. The design gives $O(1)$ insert and $O(1)$ lookup and also stores timestamps and avoids the 32 MB fixed allocation for sparse working sets.

Chunked vector (`uvm_dirty_ds_chunked.c`) Splits entries into a sequence of fixed-capacity chunks. New entries are appended in $O(1)$ to the current *active chunk* under an IRQ-safe `spinlock_t`. When the active chunk fills, it is *sealed* (a new chunk is allocated and installed with a `WRITE_ONCE` on `num_chunks`) and a background `kthread` (`uvm_dirty_sorter`) is woken via a wait queue to sort it using the kernel's `sort()`. Once a chunk is sorted, its `sorted` flag is published with `smp_store_release` so readers can use it via `smp_load_acquire` without taking the spinlock. Lookup and `for_each_in_range` for sealed sorted chunks use binary search; for the unsorted active chunk they fall back to a linear scan under the spinlock. This design defers the $O(n \log n)$ sort cost to a background thread, keeping the fault-handler insert path at $O(1)$ amortised while still providing sorted output for the dump path. Up to `MAX_CHUNKS` = 64 chunks are supported; the kthread is stopped on `destroy`.

Concurrency Design

The dirty-tracking pipeline spans two execution contexts that run concurrently: (1) the GPU replayable-fault bottom-half thread that calls `uvm_dirty_page_table_record` on every qualifying write fault, and (2) user-space procfs callers that drive lifecycle transitions. Six distinct synchronization primitives coordinate these contexts.

1. `uvm_dirty_ds_rwsem` - table read/write semaphore Declared with `DECLARE_RWSEM` in `uvm_common.c`, this kernel read-write semaphore is the primary guard for the three global dirty-set table pointers (`g_dirty_ds`, `g_dirty_ds_snapshot`, `g_dirty_ds_cumulative`) and for the session-metadata fields (`g_dirty_owner_tgid`, `g_snapshot_pending`, `g_dirty_epoch`).

- **Read side (shared).** All hot-path operations take the read side so that multiple CPUs can record dirty pages concurrently without blocking each other. Specifically, `uvm_dirty_page_table_record` (fault-handler insert), `uvm_dirty_page_table_lookup`, `uvm_dirty_tracking_started`, and `uvm_owner_tgid_with_dirty_tracking_started/ active` all acquire `down_read`. The dump handler's iteration over the query table (`uvm_dirty_ds_for_each_in_range`) also runs under `down_read`, preventing a concurrent cutover from swapping the pointer mid-iteration.
- **Write side (exclusive).** Only three operations require exclusive access: table initialisation (`uvm_dirty_page_table_init`), table destruction (`uvm_dirty_page_table_destroy`), and the cutover pointer swap inside `uvm_dirty_new_snapshot`. Each holds `down_write` only for the brief critical section during which the live or snapshot pointer is replaced; insert-path threads block for this interval and then resume on the new table.

Replacing a simple mutex with an rwsem here was a deliberate performance choice: the fault-handler path can be invoked concurrently from multiple CPU cores, and taking an exclusive lock per insert would serialise all of them. Under the rwsem, inserts from N cores proceed in parallel; the write side is held only for the $O(1)$ pointer swap during cutover.

2. `uvm_dirty_lifecycle_lock` - procfs serialisation mutex Declared with `DEFINE_MUTEX` in `uvm_common.c`, this mutex ensures that at most one procfs lifecycle operation is in progress at any time. All eight procfs handlers acquire it on entry and release it on return, including `start`, `stop`, `pause`, `resume`, `query_cutover`, and `query_dump`. Without this mutex, two concurrent `start` calls could both pass the "not started" guard and double-initialise tables; a concurrent `stop` and `cutover` could destroy tables that `cutover` is mid-swap. The mutex also serialises multi-step operations like `start` (which calls init, then downgrade, then activate, then barrier-end as a sequence) against any interleaving call.

3. `g_uvm_global.va_spaces.lock` - global VA-space list mutex This pre-existing UVM mutex protects the global linked list of registered `uvm_va_space_t` objects. The dirty-tracking code acquires it in two places:

- **Start barrier** (`uvm_dirty_barrier_begin`). Held while scanning the list to locate the unique `va_space` whose `owner_tgid` matches the tracking target. It is kept held throughout the downgrade walk and released only by `uvm_dirty_barrier_end` after `active` has been set to `true`. This prevents a concurrent `uvm_open` or GPU registration from inserting a new `va_space` for the same TGID between the ownership check and the permission walk.
- **Query barrier** (`uvm_dirty_query_barrier_begin`). Held while scanning for the owner `va_space` and while acquiring per-GPU ISR locks (see below). Released by `uvm_dirty_query_barrier_end` after all ISR locks have been dropped.

4. `va_space->lock` - per-VA-space reader/writer semaphore Each `uvm_va_space_t` carries an rwsem wrapped behind `uvm_va_space_down_write` / `uvm_va_space_down_read`. The write side additionally acquires two helper mutexes (`serialize_writers_lock` and `read_acquire_write_release_lock`) to implement priority inversion avoidance. The dirty-tracking code uses this lock in two modes:

- **Write mode - start/resume downgrade.** `uvm_dirty_downgrade_all_permissions` calls `uvm_va_space_down_write` (after `g_uvm_global.va_spaces.lock` is already held). The write lock excludes all fault-service threads, which hold the read side while processing faults for that VA space. The permission walk and the subsequent `active=true` write both occur while the write lock is held, closing both Race A (activate before downgrade) and Race B (downgrade before activate) described in the Determinism section.
- **Read mode - query barrier GPU enumeration.** `uvm_dirty_query_barrier_begin` takes `uvm_va_space_down_read` to iterate the set of registered GPUs safely before locking per-GPU ISR state. The read lock is held until the ISR locks have been acquired and the fault buffer is confirmed empty, then released by `uvm_dirty_query_barrier_end`.

5. `va_block->lock` - per-VA-block mutex Each `uvm_va_block_t` carries a `uvm_mutex_t` that protects its PTE bitmasks and GPU state. `uvm_dirty_downgrade_all_permissions` acquires this lock for each block while it snapshots the per-page write/atomic permission bitmasks into `dirty_downgraded_pages` and `dirty_downgraded_atomic_pages`, issues `uvm_va_block_revoke_prot_mask`, and calls `uvm_tracker_wait` to confirm GPU TLB invalidation. The block lock is taken only while processing that block and released before moving to the next; it is nested inside the `va_space` write lock.

6. Per-GPU ISR lock - replayable-fault service lock `uvm_parent_gpu_replayable_faults_isr_lock` acquires the ISR service lock (`parent_gpu->isr.replayable_faults.service_lock`) from a non-interrupt context. To prevent an interrupt storm, taking this lock also disables the hardware replayable-fault interrupt for that GPU; fault interrupts are re-enabled when the lock is released by the corresponding unlock call.

This lock is used exclusively in the **query barrier** (`uvm_dirty_query_barrier_begin`). After taking `va_spaces.lock` and `va_space_down_read`, the barrier iterates all parent GPUs registered to the owner VA space and acquires the ISR lock for each. With all ISR locks held, no new fault bottom-half can be scheduled. The barrier then spins on `uvm_parent_gpu_replayable_faults_pending` until the fault buffer is drained, confirming that every in-flight fault has completed service. Only after this confirmation does the caller proceed to swap tables (cutover) or disable recording (pause), ensuring a precise, race-free boundary.

7. `WRITE_ONCE` / `READ_ONCE` on `g_uvm_dirty_tracking_active` The boolean flag `g_uvm_dirty_tracking_active` is the hottest read in the entire pipeline: the fault handler checks it on every qualifying write fault before deciding whether to insert a dirty record. Rather than taking any lock for this check, the flag is accessed through `READ_ONCE` in `uvm_dirty_tracking_active()` and written through `WRITE_ONCE` in `uvm_dirty_set_tracking_active()` and `uvm_dirty_page_table_destroy()`. These barriers prevent

the compiler from caching, tearing, or reordering the access. Correctness of the surrounding critical sections is guaranteed by the query barrier or the `va_space` write lock held at the moment `WRITE_ONCE` executes; `READ_ONCE` provides only visibility, not ordering.

8. Backend-level locks Two backends introduce additional internal locking beneath the rwsem:

- **Unsorted-vector spinlock.** `uvm_dirty_ds_vector_unsorted.c` embeds a `spinlock_t` in the vector struct, acquired with `spin_lock_irqsave` / `spin_unlock_irqrestore` around each append, lookup, and iteration. The IRQ-save variant is necessary because this backend is called from the fault bottom-half, which can run with local interrupts enabled; a plain spin lock would deadlock if an interrupt preempted the lock holder and tried to re-acquire it.
- **xarray atomic compare-and-swap.** `uvm_dirty_ds_xarray.c` uses `xa_cmpxchg` with `GFP_ATOMIC` for first-write-wins insert. The xarray data structure maintains its own internal spinlock; `xa_cmpxchg` performs a single atomic conditional store under that lock without requiring the caller to take any additional lock. A non-`NULL` return value indicates the page was already present (a later write), and the allocation is freed; a `NULL` return confirms the entry was inserted as the first write.

Lock ordering summary To prevent deadlock, all code that acquires multiple locks does so in the following fixed order (outer to inner):

```
lifecycle_lock → va_spaces.lock → va_space.lock
                                                → ISR locks → va_block.lock → ds_rwsem
```

The fault handler (bottom-half context) enters only from `va_space.lock` (read) downward and never acquires `lifecycle_lock` or `va_spaces.lock`, so it cannot participate in a cycle with the lifecycle path. The dump handler holds `lifecycle_lock` and `ds_rwsem` (read) simultaneously but never acquires `va_spaces.lock`, keeping it disjoint from the downgrade path.

Prefetching

Prefetch faults are treated as non-dirtying events in the tracking pipeline. Hardware prefetch accesses are decoded as `UVM_FAULT_ACCESS_TYPE_PREFETCH`, while dirty insertion in `uvm_gpu_replayable_faults.c` is gated by `service_access_type >= UVM_FAULT_ACCESS_TYPE_WRITE` together with `uvm_dirty_tracking_active()`. Therefore, prefetch/read faults never insert entries into the dirty table directly. This includes access-counter-driven paths, which are also serviced as prefetch-class accesses in UVM's fault/migration pipeline.

To prevent prefetch from silently restoring writable mappings, the permission path in `compute_new_permission()` (`uvm_va_block.c`) clamps read/prefetch promotion paths back to `UVM_PROT_READ_ONLY` for the tracked owner. This preserves first-write capture by ensuring that a later true write still faults before any dirty insertion occurs.

The migration path is also constrained. In `block_migrate_map_unmapped_pages()` (`uvm_migrate.c`), GPU-destination pages are mapped as `READ_ONLY` while tracking is started. This closes the `cudaMemPrefetchAsync`-style bypass in which unmapped pages could otherwise become writable without passing through normal replayable-fault permission computation.

Finally, if a prefetch request violates logical permissions, replayable-fault servicing marks it as an invalid prefetch (`mark_fault_invalid_prefetch`) instead of routing it through fatal write/atomic cancellation. This keeps forward progress intact while maintaining strictly write-triggered dirty accounting.

Testing and Analysis

The testing and analysis was done in two parts

- **Correctness Testing**
- **Performance Testing**

Correctness Testing

The correctness was evaluated using CUDA tests under `correctness_tests`. The suites are intended to cover the following categories :

1. `access_type_filtering`
   Tests that only writes (including atomic read-modify-write operations) are recorded as dirty, while pure reads leave no trace. Also verifies that prefetched pages are properly tracked and that distinct access types to the same page are correctly de-duplicated.

2. `address_range_filtering`
   Verifies that dirty tracking correctly isolates multiple non-contiguous allocations, handles concurrent multi-threaded writes to distinct sub-ranges, and produces page-aligned dump entries even when GPU accesses are at sub-page granularity. Also tests that inverted or zero-length range arguments do not suppress entries, and that pages whose PTEs were established before `start_track` are still captured when written after tracking begins.

3. `backend_correctness`
   Validates fundamental invariants of the tracking system, all dump entries must be page-aligned with no duplicates, sparse write patterns must not produce phantom entries, and timestamps within and across epochs must be non-decreasing. Also verifies that freeing an allocation does not erase its tracking records, a page allocated after tracking is still tracked, and that empty epochs between writes yield empty dumps.

4. `cumulative_mode`
   Confirms that cumulative mode accumulates pages across epochs (all pages from prior windows remain visible in subsequent dumps) and that re-writing an already-tracked page does not create a duplicate or update its timestamp (first-write-wins semantics hold across epochs). A stop/restart in delta mode is also verified to reset the tracking state completely.

5. `determinism`
   Tests the cutover border: pages written before cutover must appear in the resulting snapshot while pages written after must not, even when the two write phases are interleaved closely in time. Also tests pause/resume semantics, verifying that writes issued during a pause are not recorded and that tracking resumes cleanly after `resume_track`.

6. `first_write_wins`
   Verifies the *first write wins policy*, re-writing the same page must not update its timestamp or produce a duplicate entry. This is tested both for sequential re-writes and for 64 concurrent CUDA streams racing to write the same page simultaneously. Also confirms that after cutover (without PTE re-invalidation), a second write to the same page does not re-appear in the next epoch.

7. `footprint_clearing`
   Checks that stopping tracking fully restores page permissions: pages that were READ_WRITE before tracking must be READ_WRITE again, and pages that had no prior GPU mapping must not be spuriously granted READ_WRITE on stop. Since there is no user space API to verify this this is done via write latency. Idempotency across multiple consecutive start/stop cycles is also verified.

8. `lifecycle_races`
   Stress-tests concurrent and destructive lifecycle transitions: stop while a GPU kernel is writing, rapid start/stop cycling, two processes attempting simultaneous tracking (second must get `EBUSY`), owner process death (next process must recover cleanly), `stop` while a dump is being read, `pause` with in-flight faults, and `resume` correctly re-downgrading PTEs.

9. `ordering`
   Validates temporal ordering guarantees, writes before `start_track` must not appear in the dump, timestamps must be monotonically non-decreasing, queries in cumulative mode must be non-destructive, and epoch isolation must hold. Also tests concurrent CUDA streams and high-volume sentinel/flood write patterns to verify timestamp ordering under load.

10. `procfs_robustness`
    Tests error handling on invalid procfs interactions: double `start`, `stop` without `start`, stopping with a wrong PID, malformed input strings, double cutover without an intervening dump, cutover or dump without an active session and changing the hint during an active tracking session.
    All calls must return well-defined errors without crashing or corrupting driver state.

11. `scale`
    Exercises the backing data structure at scale, 100K page single epoch writes, incremental growth across many cutover/dump cycles, small chunk procfs reads of large dumps, and writes near the maximum span of a bitmap based backend to catch off by one or capacity errors.

12. `snapshot_isolation`
    Verifies snapshot atomicity, pages written after cutover must not appear in the current snapshot, empty epochs must not carry over pages from the previous epoch, concurrent inserts during a dump read must not corrupt the snapshot, and a second read of an already drained delta mode dump must return zero entries.

13. `stats_accuracy`
    Validates the diagnostic stats interface, the insert counter must match the number of distinct pages written, counters must reset to zero on stop/start, lock-wait counters must be non-negative, and stats must be disabled (zeroed or unavailable) by default.

14. `table_lifecycle`
    Tests the dirty-tracking table through basic create/destroy cycles, verifying that reinitialisation clears all entries, that stop/start produces an empty table, that multiple reinit cycles are stable under stress, and that two independent allocations tracked in one session both appear correctly in the dump.

15. `tracking_hints`
    Tests the hint interface for selecting the data structure backend, verifies that on using either hint, the output is consistent with the expected one, and that switching hits bwtween sessions does not interfere with the tracking state.

16. `generic`
    A broad integration suite covering the core procfs interface (`dirty_tracking_start/stop/query_cutover/query_dump`). Validates that writes are tracked, reads are not, cutover atomically snapshots the epoch, stop fully resets state, `cudaFree` does not remove entries, and cumulative vs. delta mode behave as specified.

Observations The tests were run as root, and the current implementation passed every test that is present in the repository. The aggregate result was **74/74 passed**: `table_lifecycle` 5/5, `ordering` 8/8, `determinism` 2/2, `first_write_wins` 3/3, `cummulative_mode` 4/4, `address_range_filtering` 5/5, `access_type_filtering` 4/4, `snapshot_isolation` 4/4, `backend_correctness` 9/9, `procfs_robustness` 8/8, `lifecycle_races` 7/7, `scale` 4/4, `stats_accuracy` 4/4, `footprint_clearing` 3/3, `tracking_hints` 3/3, and the `generic` suite 1/1. The per-test runners for the suites report runtimes between 1.95 s and 3.05 s.

Suite	Test	Purpose	Result	Time
Table Lifecycle	`tc01_basic_lifecycle`	Basic start→write→cutover→dump→stop cycle	PASS	2.00s
Table Lifecycle	`tc02_reinit_clears_entries`	Reinit clears all tracked entries	PASS	1.99s
Table Lifecycle	`tc03_stop_start_empty_table`	stop+start yields an empty table	PASS	1.95s
Table Lifecycle	`tc04_multi_reinit_stress`	Multiple reinit cycles are stable under stress	PASS	2.01s
Table Lifecycle	`tc05_two_allocs_one_table`	Two allocations both appear in one session's dump	PASS	1.95s
Ordering	`tc01_write_before_start`	Pre-start writes do not appear in any dump	PASS	2.00s
Ordering	`tc02_ts_monotonic`	Timestamps are monotonically non-decreasing	PASS	2.01s

Continued on next page

Suite	Test	Purpose	Result	Time
Ordering	`tc03_query_nondestructive`	Cumulative dump reads are non-destructive	PASS	2.00s
Ordering	`tc04_empty_table_active`	Active session with no writes returns empty dump	PASS	2.00s
Ordering	`tc05_epoch_isolation`	Each epoch's dump contains only that epoch's writes	PASS	2.02s
Ordering	`tc06_concurrent_streams`	Timestamps ordered correctly across concurrent streams	PASS	1.99s
Ordering	`tc07_sentinel_flood`	Sentinel page timestamp precedes 1M-thread flood	PASS	1.95s
Ordering	`tc08_flood_between_queries`	Phase-A max timestamp $\leq$ phase-B min timestamp	PASS	1.98s
Determinism	`tc01_cutover_dump_border`	Pre-cutover pages in snap1; post-cutover pages in snap2	PASS	1.99s
Determinism	`tc02_pause_resume_semantics`	Paused writes not recorded; resume re-arms tracking	PASS	2.02s
First Write Wins	`tc01_same_page_repeat_write`	Re-writing a page preserves original timestamp	PASS	1.99s
First Write Wins	`tc02_concurrent_same_page`	64 streams racing on one page produce exactly 1 entry	PASS	2.00s
First Write Wins	`tc03_redirty_after_cutover`	Post-cutover re-write does not appear in next epoch	PASS	2.02s
Cumulative Mode	`tc01_cumulative_basic`	Epoch 2 dump contains all pages from both epochs	PASS	2.03s
Cumulative Mode	`tc02_cumul_vs_delta_consist.`	Each delta dump is a subset of the cumulative dump	PASS	2.44s
Cumulative Mode	`tc03_cumul_redirty_idempotent`	Re-writing a tracked page produces no duplicate	PASS	2.10s
Cumulative Mode	`tc04_cumul_stop_start_accum.`	Stop+restart resets state completely in delta mode	PASS	2.00s
Addr. Range Filtering	`tc01_two_alloc_range_isolation`	Two allocations are isolated in a single session	PASS	1.99s
Addr. Range Filtering	`tc02_range_seq_reads_threaded`	Concurrent threaded writes to distinct sub-ranges	PASS	2.03s
Addr. Range Filtering	`tc03_subpage_start_rounding`	Sub-page accesses produce page-aligned dump entries	PASS	2.05s
Addr. Range Filtering	`tc04_inverted_and_zero_range`	Inverted/zero-length range does not suppress entries	PASS	2.06s
Addr. Range Filtering	`tc05_filter_before_start_track`	Pages with pre-start PTEs still tracked when written	PASS	2.07s
Access Type Filtering	`tc01_read_only_no_dirty`	Read-only GPU access produces no dirty entries	PASS	2.05s
Access Type Filtering	`tc02_atomic_does_dirty`	Atomic RMW records the page as dirty	PASS	2.00s
Access Type Filtering	`tc03_read_then_write_distinct`	Read half leaves no trace; write half is tracked	PASS	2.00s
Access Type Filtering	`tc04_prefetch_no_dirty`	Prefetched pages are not recorded as dirty	PASS	2.01s
Snapshot Isolation	`tc01_post_cutover_writes_excl.`	Post-cutover writes excluded from current snapshot	PASS	2.01s
Snapshot Isolation	`tc02_empty_epoch_no_carryover`	Empty epoch yields no carryover from previous epoch	PASS	2.07s
Snapshot Isolation	`tc03_conc._insert_during_dump`	Concurrent inserts do not corrupt snapshot being read	PASS	2.17s
Snapshot Isolation	`tc04_multiple_readers_same_snap.`	Second read of drained delta dump returns 0 entries	PASS	2.02s
Backend Correctness	`tc01_all_entries_page_aligned`	All dump entries are page-aligned, no duplicates	PASS	2.03s

Continued on next page

Suite	Test	Purpose	Result	Time
Backend Correctness	`tc02_sparse_write_pattern`	Sparse writes produce no phantom entries	PASS	2.01s
Backend Correctness	`tc03_address_monotonic_in_dump`	Reports whether dump output is address-monotonic	PASS	1.96s
Backend Correctness	`tc04_multiple_allocs_independent`	Multiple allocations tracked independently, no aliasing	PASS	2.03s
Backend Correctness	`tc05_invalidate_doesnt_clear`	`cudaFree` does not remove tracking entries	PASS	2.01s
Backend Correctness	`tc06_ts_monotonic_within_epoch`	Timestamps non-decreasing within a single epoch	PASS	2.00s
Backend Correctness	`tc07_cross_epoch_ts_ordering`	Max ts of epoch $N \leq$ min ts of epoch $N+1$	PASS	2.01s
Backend Correctness	`tc08_empty_epoch_between_writes`	Empty epoch between writes yields empty dump	PASS	2.02s
Backend Correctness	`tc09_alloc_after_start_tracked`	Post-start allocation captured when written	PASS	2.04s
Procfs Robustness	`tc01_double_start`	Second `start` returns EBUSY; state intact	PASS	2.03s
Procfs Robustness	`tc02_stop_without_start`	`stop` without prior start returns error	PASS	2.00s
Procfs Robustness	`tc03_wrong_pid_stop`	Wrong-PID stop returns error; own session unaffected	PASS	1.97s
Procfs Robustness	`tc04_malformed_start_write`	Malformed start strings return EINVAL	PASS	1.99s
Procfs Robustness	`tc05_double_cutover_no_dump`	Second cutover returns EBUSY; snapshot preserved	PASS	1.99s
Procfs Robustness	`tc06_cutover_without_start`	Cutover/dump without active session return errors	PASS	2.01s
Procfs Robustness	`tc07_hint_invalid_input`	Invalid hint input returns EINVAL	PASS	2.01s
Procfs Robustness	`tc08_hint_busy_during_tracking`	Trying to write to `dirty_tracking_active` during tracking returns EBUSY	PASS	2.03s
Lifecycle Races	`tc01_stop_while_kernel_running`	`stop` during active GPU write does not crash	PASS	1.99s
Lifecycle Races	`tc02_rapid_start_stop_cycle`	Rapid start/stop cycles leave no leaks or corruption	PASS	2.00s
Lifecycle Races	`tc03_conc._start_two_processes`	Second process attempting start receives EBUSY	PASS	1.95s
Lifecycle Races	`tc04_owner_process_dies`	New session works after owner process is killed	PASS	2.04s
Lifecycle Races	`tc05_stop_while_dump_reading`	`stop` during slow dump read does not deadlock	PASS	3.05s
Lifecycle Races	`tc06_pause_with_inflight_faults`	Pause stops recording; resume re-enables it	PASS	1.99s
Lifecycle Races	`tc07_resume_re_downgrade`	Resume re-downgrades PTEs so writes are re-tracked	PASS	2.01s
Scale	`tc01_large_page_count`	100K pages tracked in one epoch without truncation	PASS	2.29s
Scale	`tc02_repeated_growth_and_query`	Incremental growth over many cutover/dump cycles	PASS	2.04s
Scale	`tc03_procfs_paginated_read`	Small-chunk dump reads produce correct output	PASS	2.01s
Scale	`tc04_near_bitmap_max`	Full-span write near bitmap backend capacity limit	PASS	2.00s
Stats Accuracy	`tc01_insert_count_matches_pages`	Insert counter equals number of distinct pages written	PASS	2.02s
Stats Accuracy	`tc02_stats_reset_on_reinit`	Stats counters reset to zero on stop+start	PASS	2.01s
Stats Accuracy	`tc03_lock_wait_nonneg`	Lock-wait counter is non-negative after a session	PASS	1.96s

Continued on next page

Suite	Test	Purpose	Result	Time
Stats Accuracy	`tc04_stats_disabled_by_default`	Stats disabled by default; toggle required	PASS	1.97s
Footprint Clearing	`tc01_rw_restored_after_stop`	Pre-tracking RW pages restored to RW after stop	PASS	1.99s
Footprint Clearing	`tc02_fresh_pages_still_fault`	Unmapped pages not spuriously granted RW on stop	PASS	2.02s
Footprint Clearing	`tc03_idempotent_cycles`	Multiple start/stop cycles each leave pages in RW state	PASS	1.95s
Tracking Hints	`tc01_hint_seq_session`	Giving hint as `WRITE_SEQ` all the writes are recorded	PASS	2.00s
Tracking Hints	`tc02_hint_rand_session`	Giving hint as `WRITE_RAND` all the writes are recorded	PASS	1.98s
Tracking Hints	`tc03_hint_switch_between_sessions`	Switching hints in between tracking sessions does not raise any errors	PASS	1.97s
Generic	`test_dirty_tracking_suite`	Integration smoke test covering all core behaviors	PASS	2.02s

Performance Testing

The performance overhead of the dirty-tracking pipeline was evaluated using the `run_overhead_benchmark.sh` script, which measures wall-clock time for each benchmark with tracking **OFF** (baseline) and tracking **ON**, then reports average wall time, standard deviation, and overhead percentage in a CSV. Each benchmark is repeated `REPS` times (default: 5) and the results are averaged. Seven benchmarks spanning write-intensive, compute-bound, read-heavy, and mixed access patterns were selected: `int_set_uvm` (4K-stride and sequential), polybench GEMM, 2MM, BICG, and MVT, and Rodinia Needle (Needleman–Wunsch).

Memory Configurations Each benchmark is run across eight working-set sizes that span both the *in-GPU* regime and the *oversubscribed* regime:

- **In-GPU** (fits in VRAM of the test A40, ≈45 GB): 512 MB, 4 GB, 8 GB, 16 GB, 32 GB, 40 GB.
- **Oversubscribed** (exceeds GPU VRAM, triggers UVM eviction to host): 48 GB, 64 GB.

Benchmarks

1. **`int_set_uvm` (4K stride)**
   Worst-case write-intensive synthetic: every write touches a fresh page at 4 KB stride, maximising UVM fault events and dirty-tracking insertions per byte written. Run across both in-GPU and oversubscribed configurations to measure how overhead scales with working-set size and to observe eviction-driven re-faulting behavior.
2. **`int_set_uvm` (sequential)**
   Control for the 4K-stride variant: coalesced sequential writes to the same allocation generate fewer fault events per byte, isolating per-fault tracking overhead from per-byte overhead. Compared directly with the stride variant to quantify the impact of access pattern on tracking cost across identical working-set sizes.
3. **polybench GEMM**
   Compute-bound reference: high arithmetic density amortises tracking cost over long kernel runtimes, measuring overhead on kernels dominated by floating-point computation rather than memory faults. Also run in the oversubscribed regime; a parameter-driven regime shift at large matrix sizes causes baseline time to drop sharply, making the fixed tracking overhead appear proportionally larger.
4. **polybench 2MM**
   Chained dual-GEMM benchmark: two consecutive matrix multiplications produce a larger write footprint across two output matrices, testing tracking-table scalability under heavier write load than a single GEMM while retaining high arithmetic intensity. Oversubscribed configurations were not collected for this benchmark.
5. **polybench BICG**
   Read-heavy workload: predominantly read traffic exercises the tracking pipeline's read-fault bypass path

and measures overhead on kernels with a minimal write footprint. Also run in the oversubscribed regime, where the fraction of time spent in fault handling increases and amplifies per-insertion cost.

6. **polybench MVT**
   Read-mostly with a small write component: verifies that a minor write footprint is correctly tracked without inflating overhead on otherwise read-heavy traffic. Complements BICG with a slightly different read-to-write ratio and is also exercised in the oversubscribed regime.
7. **Rodinia Needle (Needleman–Wunsch)**
   Real-world dynamic-programming kernel with mixed read/write access to a 2-D score matrix; working-set size is controlled via the `-mb` parameter. Above a working-set threshold the benchmark reduces its iteration count, drastically shrinking the write footprint and the number of UVM faults fired—causing overhead to collapse from >800% at small sizes to near zero at large ones.

Results Table 3 reports measured overhead percentages for all benchmark/configuration pairs (5 repetitions each, averaged). The shaded rows mark the two oversubscribed configurations (48 GB and 64 GB) where the working set exceeds GPU VRAM and UVM must evict pages to host memory.

Size	4K stride	Sequential	GEMM	2MM	BICG	MVT	Needle
512 MB	+57.5%	+15.7%	+26.0%	+21.5%	+26.9%	+31.7%	+113.9%
4 GB	+372.3%	+163.4%	+19.3%	+67.2%	+131.1%	+131.7%	+871.0%
8 GB	+659.9%	+302.9%	+15.1%	+39.6%	+171.3%	+153.1%	+959.8%
16 GB	+1108.2%	+457.9%	+13.0%	+11.2%	+191.0%	+178.4%	+871.5%
32 GB	+1537.6%	+630.0%	+207.6%	+9.8%	+257.9%	+265.2%	+0.6%
40 GB	+1688.3%	+703.2%	+258.8%	+9.1%	+264.6%	+257.0%	+19.1%
48 GB†	+1730.7%	+699.4%	+213.1%	—	+263.6%	+290.6%	+27.8%
64 GB†	+1547.1%	+648.6%	+261.6%	—	+208.4%	+253.5%	+11.8%

Table 3: Dirty-tracking overhead percentage by benchmark and working-set size (tracking ON vs. OFF wall time, averaged over 5 runs). Column headers *4K stride* and *Sequential* refer to the two `int_set_uvm` variants. † Oversubscribed. 2MM oversubscribed runs were not collected.

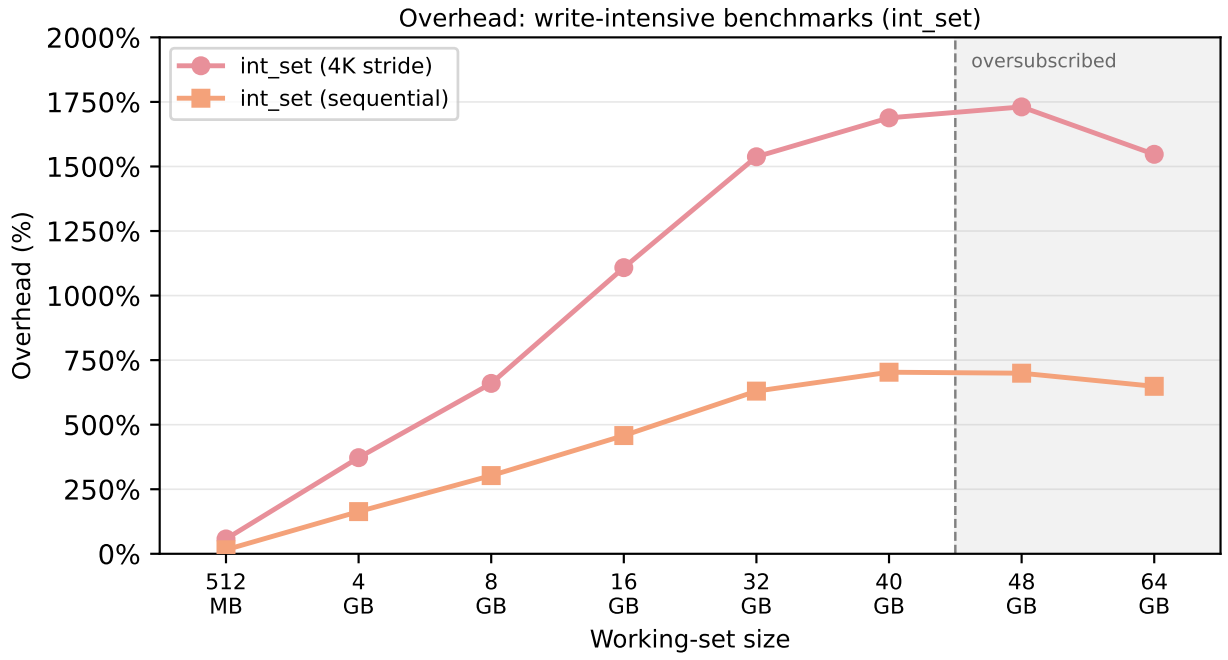

Figure 3: Overhead for write-intensive `int_set_uvm` benchmarks. The 4K-stride variant maximises UVM fault events per byte written, producing the highest overhead of any benchmark tested. Sequential access generates fewer faults per byte and achieves 2-8× lower overhead at comparable sizes. Grey shaded region: oversubscribed regime (≥48 GB).

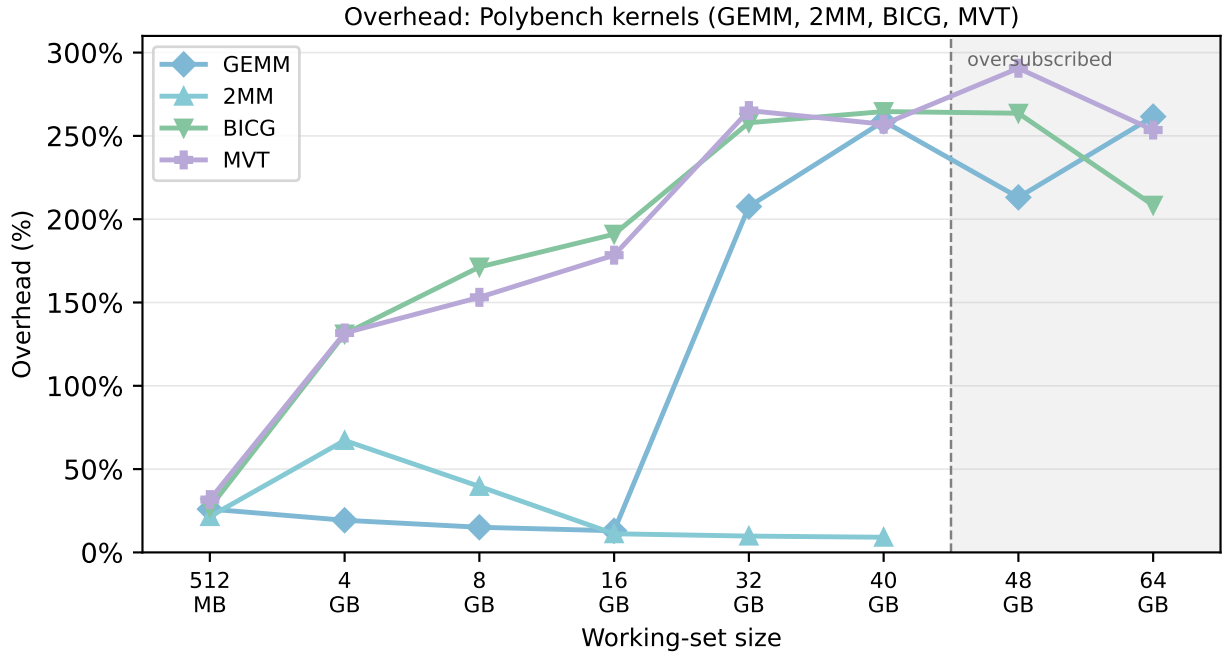

Figure 4: Overhead for Polybench kernels. GEMM and 2MM show low overhead while the working set fits in VRAM, as compute time dominates fault-handling cost. BICG and MVT are read-heavy but still accumulate non-trivial overhead from their write footprints and initial page-population faults. Grey shaded region: oversubscribed regime.

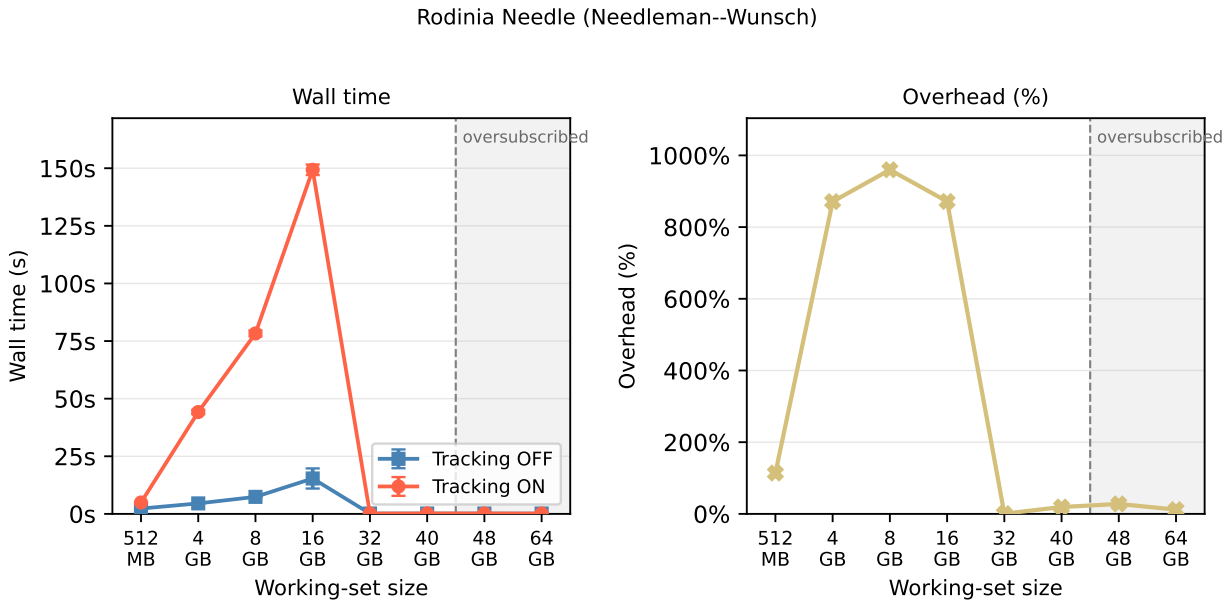

Figure 5: Rodinia Needle (Needleman-Wunsch). *Left:* wall-time comparison (tracking OFF vs. ON) with error bars (± 1 standard deviation over 5 runs). *Right:* derived overhead percentage. Overhead is very high at 4-16 GB (871-960%) and collapses near zero at $\geq$32 GB due to an algorithmic regime shift in the benchmark at that working-set size.

Analysis of Results

Write-intensive workloads. The `int_set_uvm` benchmarks represent the worst case for the tracking pipeline: every write touches a fresh page, so the number of dirty-tracking insertions equals the total page count. With 4K-stride access, overhead scales from +57.5% at 512 MB to +1730.7% at 48 GB because a larger working set generates proportionally more UVM fault events, each carrying a tracking insertion. Sequential access produces the same number of dirty pages but consolidates memory traffic so that fewer fault events fire per byte written, reducing overhead by 2-8× at comparable sizes (Figure 3). In the oversubscribed regime (≥48 GB) pages are repeatedly evicted and re-faulted, but first-write-wins semantics within each epoch cap insertions to one per page regardless of eviction frequency. The slight drop in 4K-stride overhead from 48 GB (+1730.7%) to 64 GB (+1547.1%) reflects that at 64 GB baseline execution time grows faster than tracking overhead, as the eviction subsystem dominates baseline performance.

Compute-bound workloads (GEMM, 2MM). When the working set fits in VRAM, arithmetic intensity dominates and tracking adds only 13-26% for GEMM. At ≥32 GB the benchmark's matrix-dimension parameter changes and the baseline execution time drops sharply (205 s at 16 GB vs. 19 s at 32 GB), making the fixed tracking overhead appear proportionally larger (207-261%). 2MM achieves the lowest overhead of any benchmark at large in-GPU sizes (9.1-9.8% at 32–40 GB): two chained GEMM kernels increase total compute time while the write footprint grows only modestly.

Read-heavy workloads (BICG, MVT). Although most memory traffic is reads, both kernels show non-trivial overhead (26-31% at 512 MB, up to +290.6% in the oversubscribed regime). Read faults do not insert dirty records, but the initial page-population phase and the small write component still generate write faults. In the oversubscribed regime the fraction of time spent in fault handling increases, amplifying the per-insertion cost (Figure 4).

Rodinia Needle. Needle exhibits a two-regime pattern (Figure 5). At 512 MB-16 GB, overhead is high (113-960%): the DP score matrix generates dense write traffic, producing many first-write faults per kernel invocation. At ≥32 GB, both baseline and tracking-on execution times collapse from $O(10)$ s to $O(0.1)$ s and overhead falls to near zero (+0.6-+27.8%). This transition is caused by the benchmark's `-mb` parameter: above a working-set threshold the implementation reduces iteration count, drastically shrinking the write footprint and the number of UVM faults fired. The tracking system handles this correctly: near-zero work produces near-zero overhead.

In-GPU vs. oversubscribed summary. Across all benchmarks, the in-GPU regime shows overhead primarily proportional to write-fault density. Compute-bound kernels that amortise faults over long runtimes (GEMM at 16 GB, 2MM at 32-40 GB) achieve single-digit to low two-digit percent overhead, confirming the pipeline is lightweight relative to GPU computation. The oversubscribed regime adds eviction-driven re-faulting, but first-write-wins semantics cap dirty insertions to one per page per epoch regardless of eviction frequency, preventing unbounded overhead growth.

Query-Frequency Sensitivity Figure 6 shows how overhead varies with the number of epochs per tracking interval (1, 5, or 10 `query_dump` calls between a `start_track`/`stop_track` pair) for all 512 MB benchmarks. This experiment was run on `int_set_seq` as the primary benchmark. Each point is the mean overhead across five sleep-interval configurations (0.01 s–1.00 s), with error bars showing one standard deviation.

Across every benchmark, the three lines overlap almost entirely: adding more epochs per tracking interval does not measurably increase overhead. This is expected by design—`query_dump` in delta mode consumes the current snapshot and moves a pointer; it does not re-scan the backing structure proportional to its size. The slight spread visible on Needle reflects variability across sleep-interval settings rather than an epoch-count effect: at 512 MB the Needle working set fits comfortably in VRAM, and the overhead is governed by when the epoch boundary lands relative to active write phases rather than how many epochs are issued per tracking interval.

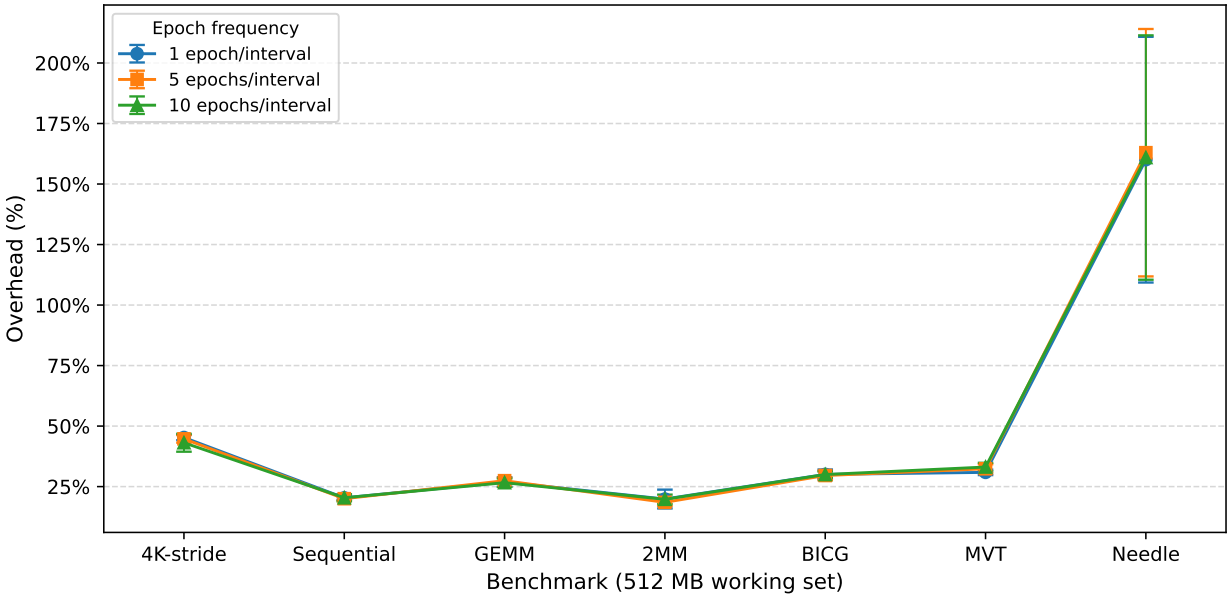

Figure 6: Overhead at 512 MB working set as a function of epoch frequency (1, 5, or 10 `query_dump` calls per tracking interval), with `int_set_seq` as the primary benchmark. Each point is the mean over five sleep-interval configurations; error bars show ±1 standard deviation. The three lines are nearly coincident for every benchmark, confirming that issuing more epochs per tracking interval does not increase tracking overhead.

Cross-Backend Overhead Comparison The six available backends differ substantially in algorithmic complexity for insert and iteration. Tables 4 and 5 compare wall-clock overhead across all backends for the two most discriminating benchmarks: `int_set_uvm` 4K-stride (maximum insert pressure) and polybench GEMM (compute-dominated, low insert pressure). Each cell is the overhead percentage (tracking ON vs. OFF, 5-run average) after recompiling the kernel module with the corresponding `.ops` pointer set in `uvm_common.c`.

Size	XArray	Vector	Vec.-U	Chunked	Nested BM	Linked List
512 MB	+36.6%	+57.5%	+900.8%	+247.9%	+48.6%	917.4%
4 GB	+272.0%	+372.3%	+27730.1%	+16951.0%	+263.6%	—
8 GB	+462.1%	+659.9%	OOM	OOM	+456.8%	—
16 GB	+680.8%	+1108.2%	OOM	OOM	+689.3%	—
32 GB	+981.1%	+1537.6%	OOM	OOM	+914.0%	—
40 GB	+1046.0%	+1688.3%	OOM	OOM	+1001.2%	—
48 GB†	+1038.7%	+1730.7%	OOM	OOM	+922.9%	—
64 GB†	+942.0%	+1547.1%	OOM	OOM	+921.0%	—

Table 4: `int_set_uvm` 4K-stride overhead by backend (5-run average, tracking ON vs. OFF). Vec.-U = Vector Unsorted; Nested BM = Nested Bitmap. Vector column carried from Table 3. † Oversubscribed.

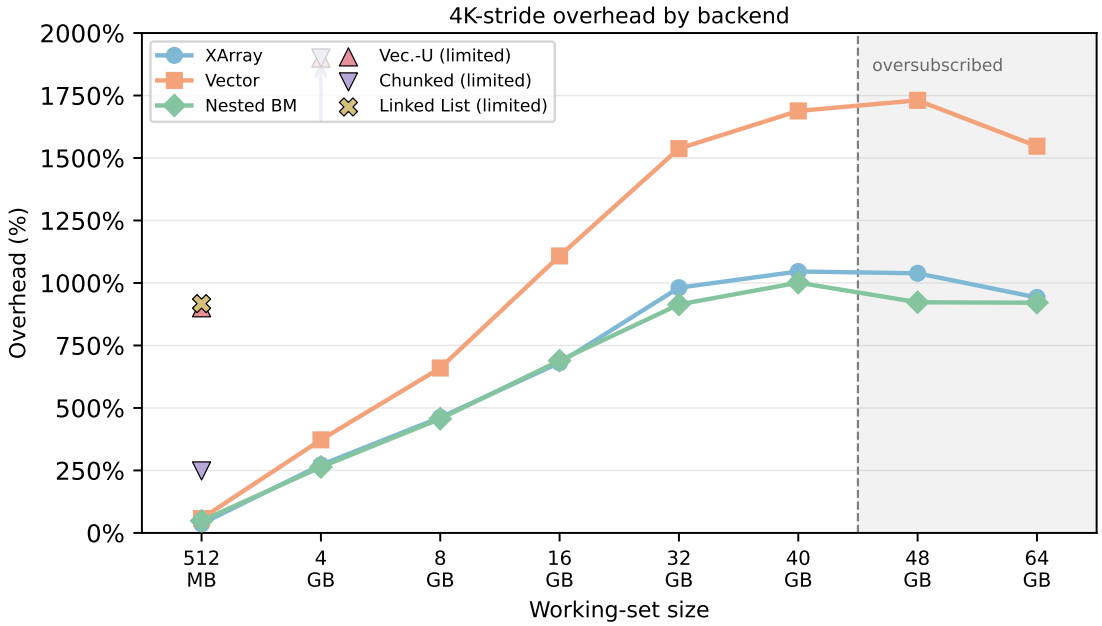

Figure 7: 4K-stride overhead by backend vs. working-set size. XArray, Vector, and Nested Bitmap scale to all sizes; Vec.-U, Chunked, and Linked List are shown only at the sizes where they did not OOM (markers with upward arrows indicate values exceeding the axis). Grey shaded region: oversubscribed regime ($\geq$48 GB).

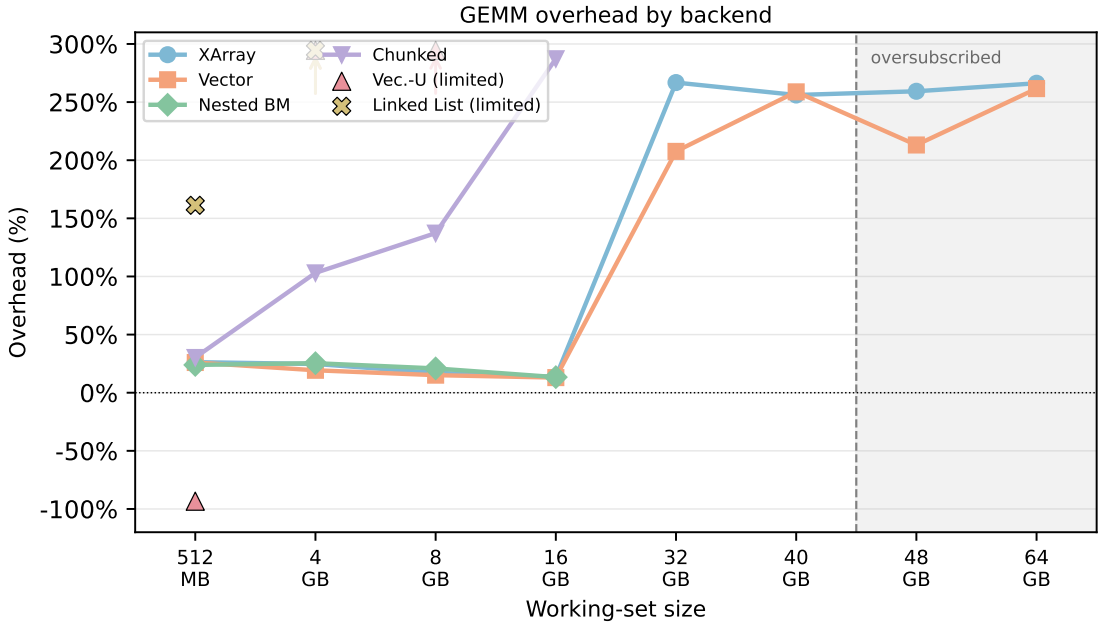

Figure 8: GEMM overhead by backend vs. working-set size. Chunked is plotted as a line up to 16 GB (its last valid size); Vec.-U and Linked List are shown as scattered markers where they fit within the axis range (upward arrow at 4 GB indicates off-chart values). Dotted horizontal line at 0% for reference. Grey shaded region: oversubscribed regime ($\geq$48 GB).

Size	XArray	Vector	Vec.-U	Chunked	Nested BM	Linked List
512 MB	+26.3%	+26.0%	−93.3%	+30.3%	+23.9%	+161.2%
4 GB	+24.7%	+19.3%	+380.8%	+103.1%	+25.4%	+1147.3%
8 GB	+17.1%	+15.1%	+551.3%	+137.3%	+20.9%	—
16 GB	+13.6%	+13.0%	OOM	+287.3%	+13.4%	—
32 GB	+266.8%	+207.6%	OOM	OOM	OOM	—
40 GB	+256.1%	+258.8%	OOM	OOM	OOM	—
48 GB†	+259.3%	+213.1%	OOM	OOM	OOM	—
64 GB†	+266.2%	+261.6%	OOM	OOM	OOM	—

Table 5: Polybench GEMM overhead by backend (5-run average, tracking ON vs. OFF). Vec.-U = Vector Unsorted; Nested BM = Nested Bitmap. Vector column carried from Table 3. † Oversubscribed.

4K-stride (Figure 7). The three scalable backends separate clearly from the rest. Nested Bitmap and XArray track nearly in lockstep at every working-set size (e.g., 263% vs. 272% at 4 GB; 922% vs. 1039% at 48 GB), reflecting their similar per-fault code paths and near-identical lock-wait times (33 ns each, Table 6). Vector runs consistently 30–70% higher than either, because each insert must maintain a sorted order via binary search (867 ns avg vs. 69–145 ns for the other two). Vec. Unsorted and Chunked exhibit catastrophic overhead at 4 GB (+27,730% and +16,951%) before OOMing; their insert latencies of 65,010 ns and 196,990 ns explain why every UVM write fault imposes an enormous tracking cost. Linked List is confined to 512 MB and shows overhead comparable to Vec. Unsorted at that size. In the oversubscribed regime (≥48 GB), Nested Bitmap and XArray converge further as eviction-subsystem cost eclipses tracking cost.

GEMM (Figure 8). The compute-bound GEMM benchmark produces far fewer write faults per second, which compresses the difference between backends. At ≤16 GB, XArray, Vector, Nested Bitmap, and Chunked all fall in the 13–30% band: kernel compute time is long enough to amortise tracking cost regardless of insert latency. Chunked's poor per-insert performance does not show at low fault counts. Vec. Unsorted shows a spurious −93.3% overhead at 512 MB (measurement noise from a very short baseline) before diverging sharply at 4–8 GB. Linked List exceeds 1,000% at 4 GB, consistent with its O(n) insert. At ≥32 GB the GEMM regime shift shrinks baseline execution time, causing all backends to spike to 207–267%; this jump is benchmark-internal and affects XArray and Vector equally.

Per-Operation Latency Across Backends Table 6 reports average insert and `for_each_in_range` latencies, lock-wait time, and total init/destroy cost for each backend, measured under the `int_set_uvm` 4K-stride workload at 4 GB (the size where insert pressure is highest without entering the oversubscribed regime; Vec. Unsorted and Linked List were measured at 512 MB due to OOM).

Backend	Insert avg (ns)	Insert lock-wait (ns)	For-each avg (s)	Init total (ns)	Destroy total (s)
XArray	145	33	0.189	466	0.098
Vector	867	35	0.052	2,881	0.034
Vec. Unsorted*	65,010	44	8.542	3,864	0.004
Chunked	196,990	43	204.234	362,386	0.044
Nested Bitmap	69	33	0.047	2,121	0.001
Linked List*	149,581	42	20.047	2,094	0.007

Table 6: Per-operation stats per backend measured via `uvm_dirty_ds_stats` under `int_set_uvm` 4K-stride at 4 GB. Init and Destroy report total ns over all calls; others report per-call averages. * Vec. Unsorted and Linked List results collected at 512 MB (OOM at 4 GB).

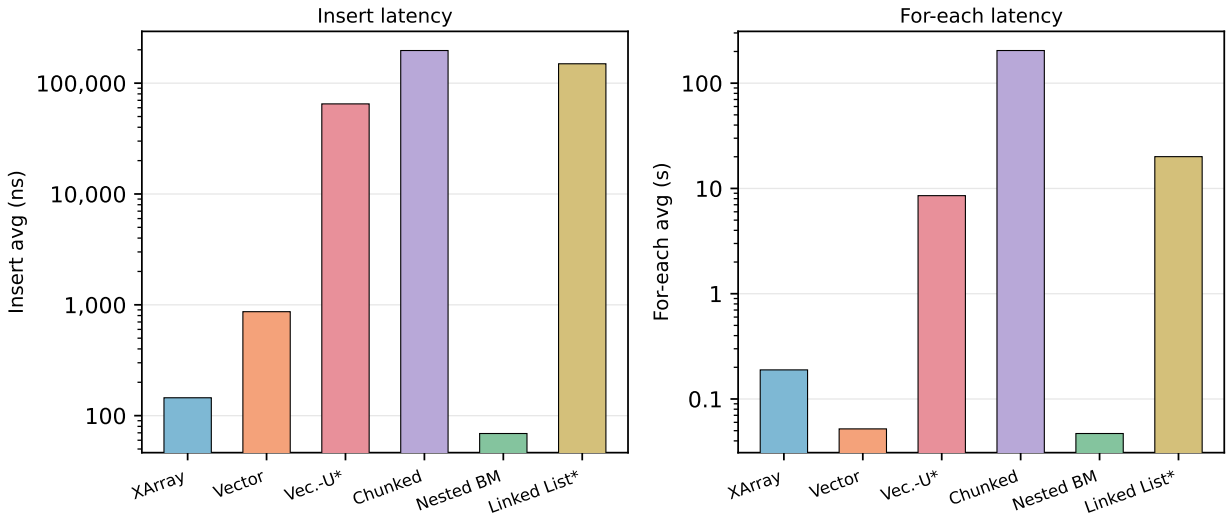

Figure 9: Insert latency (left) and for-each latency (right) per backend on a log scale. Both axes span multiple orders of magnitude; backends marked $*$ were measured at 512 MB.

Per-Operation Latency Analysis

Insert latency (Figure 9, left). Nested Bitmap achieves the lowest insert latency at 69 ns — a plain atomic bit-set into a pre-allocated bitmap — with a lock-wait of only 33 ns. XArray follows at 145 ns (sparse tree node lookup and set), also with 33 ns lock-wait. Vector is 6× slower than XArray (867 ns) due to its O(n) sorted-insert binary search. The three unscalable backends are 2–3 orders of magnitude worse: Vec. Unsorted (65,010 ns), Linked List (149,581 ns), and Chunked (196,990 ns) all incur allocation or traversal costs per insert that make them impractical under high fault rates.

For-each latency (Figure 9, right). Nested Bitmap (0.047 s) and Vector (0.052 s) are the fastest iterators: a bitmap word scan processes 64 pages per instruction, while a sorted vector offers sequential, cache-friendly iteration. XArray (0.189 s) is roughly 4× slower due to pointer chasing across sparse tree nodes. Vec. Unsorted (8.5 s) and Linked List (20 s) suffer from their unordered, pointer-linked structures. Chunked is the worst by a large margin at 204 s — over 4,000× slower than Nested Bitmap — because its chunked indirection layers must be traversed for every page during iteration. This for-each cost directly determines the wall-clock overhead seen during the `query_dump` phase, and explains why Chunked and Vec. Unsorted produce extreme overhead in the end-to-end benchmarks even at modest working-set sizes.

Future Work

Program Analysis for Backend Selection and Overhead Reduction

The largest overhead driver is the number of write faults fired per unit of work, which is determined entirely by each kernel's memory access pattern. The existing `dirty_tracking_hint` procfs entry already lets an operator select between sequential (sorted-vector) and random (nested-bitmap) backends before a session, but the selection is manual.

A natural extension is to derive the hint automatically by analysing the compiled PTX of each kernel. Store instructions (`st.*`) carry address operands whose computation reveals the access pattern: a unit-stride expression where adjacent threads write adjacent addresses signals sequential access; irregular or data-dependent address arithmetic signals random access. This classification feeds directly into the existing hint dispatch, eliminating operator involvement.

Beyond backend selection, alias analysis on the NVVM IR (LLVM dialect emitted before device linking) can identify which kernel arguments are derived from `cudaMallocManaged` pointers. At `start` and `resume`, only VA blocks reachable through those arguments need their write permissions revoked. The current implementation walks the entire owner VA space and calls `uvm_va_block_revoke_prot_mask` on every mapped block; restricting this walk to write-reachable blocks reduces the downgrade barrier latency proportionally to the fraction of the address space that is actually written. This is most valuable at large working-set sizes (32-64 GB) where the downgrade walk is a non-trivial fraction of `start`/`resume` latency.

Migration Policy for the Oversubscribed Regime

In the oversubscribed regime the GPU's on-device memory is exhausted and UVM evicts pages to host DRAM using an LRU policy that has no knowledge of write intensity. The dirty tracking data already produced by the existing implementation is sufficient to improve several aspects of this policy.

Write-heat aware eviction In cumulative tracking mode, the dump accumulates which pages have been dirtied across multiple epochs. A page that appears dirty in every epoch is rewritten every time it enters GPU memory; evicting it immediately triggers a re-fault. A page absent from the dirty set for several epochs is effectively read-only and can be evicted without a write-back. Adding an epoch-frequency counter per VA block, maintained by user space from successive dump outputs, and using it to bias UVM's eviction candidate scoring would keep write-hot pages on-device longer and preferentially evict clean pages, reducing re-fault frequency under memory pressure.

Cutover timing for iterative migration In iterative pre-copy live migration, the migration controller issues `query_cutover` to snapshot the dirty set and then transfers those pages to the destination. Convergence requires that the re-dirty rate during transfer is lower than the transfer bandwidth. The nanosecond-resolution timestamps in the dump make the write rate directly observable: the density of timestamps over the epoch's duration gives an estimate of faults per second. A migration controller that reads the insert count from `dirty_ds_stats` in real time can defer `query_cutover` until the observed write rate drops below available transfer bandwidth, improving round-trip convergence without any additional kernel instrumentation.

Dirty ratio as a scheduling signal After each dump, the number of output lines divided by the total managed allocation size gives the fraction of pages dirtied in that epoch. When this ratio is low the workload is in a read-intensive phase and migration rounds can be scheduled more aggressively; when it is high the workload is producing large dirty sets and extending the inter-round interval allows it to reach a quieter phase before the next cutover. This feedback loop is entirely implementable in user space using the existing procfs interface.

Source Code

Development Environment

Development and testing are conducted on a dedicated virtual machine (`dirtyuvm`) running:

- Linux kernel version 6.8 (custom build)
- Modified NVIDIA UVM driver version 580.65.06 (loaded as a kernel module)

Repository

The full source code is hosted at:

`https://github.com/aleph-5/open-gpu-kernel-modules`

This is a fork of NVIDIA's open-source GPU kernel modules, with our modifications applied on top.

Instructions for Run

This section gives step-by-step commands to build the module, exercise every procfs control path, and run the correctness and performance test suites.

Prerequisites

All operations require root. Verify the NVIDIA driver and CUDA toolkit are present:

```
nvidia-smi
nvcc --version
```

Build and Load the Module

```
sudo make modules -j$(nproc) # compile all kernel modules
sudo make modules_install -j$(nproc) # install .ko files
sudo modprobe -r nvidia_uvm # unload old nvidia_uvm
sudo modprobe nvidia_uvm # load modified nvidia_uvm
```

Verify the procfs entries are present:

```
ls /proc/driver/nvidia-uvm/dirty_*
```

Exercising the Procfs Interface

Replace `<PID>` with the TGID of the CUDA process under observation.

Lifecycle Controls

```
# Initialise live table, downgrade GPU PTEs to READ_ONLY, begin recording.
echo "<PID> delta" > /proc/driver/nvidia-uvm/dirty_tracking_start

# Same as above but merges every snapshot into a session-union table on dump.
echo "<PID> cumulative" > /proc/driver/nvidia-uvm/dirty_tracking_start

# Drain in-flight faults, then disable recording (tables and PTEs preserved).
echo "<PID>" > /proc/driver/nvidia-uvm/dirty_tracking_pause

# Re-downgrade PTEs and re-enable recording without reinitialising tables.
echo "<PID>" > /proc/driver/nvidia-uvm/dirty_tracking_resume

# Restore GPU permissions, destroy all tables, flush stats to /tmp/.
echo "<PID>" > /proc/driver/nvidia-uvm/dirty_tracking_stop
```

Query Controls

```
# Drain fault buffer, atomically swap live->snapshot, install fresh live table.
echo "<PID>" > /proc/driver/nvidia-uvm/dirty_tracking_query_cutover

# Emit frozen snapshot; in cumulative mode, merge into session-union first.
# Output: one line per dirty page: "0x<addr> <timestamp_ns>"
cat /proc/driver/nvidia-uvm/dirty_tracking_query_dump
```

Statistics Controls

```
# Enable instrumentation and reset all counters to zero.
echo "enable" > /proc/driver/nvidia-uvm/dirty_ds_stats_toggle

# Stop accumulation (counters preserved).
echo "disable" > /proc/driver/nvidia-uvm/dirty_ds_stats_toggle

# Print per-operation counts, average latency, rwsem wait time, callback timing.
cat /proc/driver/nvidia-uvm/dirty_ds_stats
```

Typical End-to-End Session

```
./my_cuda_workload &
PID=$!

echo "$PID delta" > /proc/driver/nvidia-uvm/dirty_tracking_start

sleep 2 # let the workload run one migration interval

echo "$PID" > /proc/driver/nvidia-uvm/dirty_tracking_query_cutover
cat /proc/driver/nvidia-uvm/dirty_tracking_query_dump > epoch1_dirty_pages.txt

echo "$PID" > /proc/driver/nvidia-uvm/dirty_tracking_pause
# ... perform stop-and-copy transfer ...
echo "$PID" > /proc/driver/nvidia-uvm/dirty_tracking_resume

echo "$PID" > /proc/driver/nvidia-uvm/dirty_tracking_query_cutover
cat /proc/driver/nvidia-uvm/dirty_tracking_query_dump > epoch2_dirty_pages.txt

echo "$PID" > /proc/driver/nvidia-uvm/dirty_tracking_stop
```

Correctness Tests

```
# Run every suite in canonical order.
sudo python3 correctness_tests/run_all.py

# Run a single suite.
cd correctness_tests/<suite_name> && sudo python3 run_tests.py

# Run specific test cases within a suite (matched by name prefix).
sudo python3 run_tests.py tc01 tc03

# Skip rebuild and re-run already-compiled binaries.
sudo python3 run_tests.py --no-build

# Show stdout even for passing tests.
sudo python3 run_tests.py --verbose
```

Performance Tests

```
# Run a single suite.
cd performance_tests/<suite_name> && sudo python3 run_tests.py

# Run specific test cases within a suite (matched by name prefix).
sudo python3 run_tests.py tc01 tc03

# Show stdout even for passing tests.
sudo python3 run_tests.py --verbose
```

Benchmarks

All benchmark commands must be run from the `cuda-oversubscribed-benchmarks-main/` directory as root. The script builds all binaries, then runs each selected benchmark `REPS` times with tracking *off* and *on*, and writes wall-time statistics and overhead percentage to a CSV.

```
# Build all benchmark binaries (also done automatically by the script).
make -j$(nproc)

# Run all benchmarks, 5 repetitions each (default).
```

```
sudo bash run_overhead_benchmark.sh

# Run all benchmarks with a custom repetition count.
sudo bash run_overhead_benchmark.sh 10

# Run a subset of benchmarks by name prefix, 5 reps.
# Valid prefixes: int_set_4k, int_set_seq, gemm, 2mm, bicg, mvt, needle
# Each prefix selects all size variants (512m, 4g, 8g, ... 64g).
sudo bash run_overhead_benchmark.sh 5 gemm bicg needle

# Write results to a custom CSV path instead of overhead_results.csv.
OUT_CSV=/tmp/my_results.csv sudo bash run_overhead_benchmark.sh
```

Results are saved to `overhead_results.csv` (columns: `benchmark`, `reps`, `off_avg_s`, `off_std_s`, `on_avg_s`, `on_std_s`, `overhead_pct`). Per-run dirty-page dumps land in `/tmp/tracking_preload_dumps/`.

Plotting Benchmark Results

The plotting script `plot_overhead.py` and requires the csv file `overhead_results.csv` as input.

The script requires `matplotlib` and `numpy`:

```
# Move into the benchmarks directory and run the script.
cd cuda-oversubscribed-benchmarks-main
pip install matplotlib numpy # if not already installed
python3 plot_overhead.py
```

Three PDF figures are written to the same directory:

```
overhead_write.pdf # int_set 4K-stride vs sequential overhead
overhead_polybench.pdf # GEMM, 2MM, BICG, MVT overhead
overhead_needle.pdf # Needle wall-time + overhead % (dual panel)
```

Each figure plots overhead percentage against working-set size (512 MB – 64 GB). A shaded region marks the oversubscribed regime (working set exceeds GPU memory).

Backend Comparison

Recompile once per backend (set `.ops` in `uvm_common.c`, then `sudo ./compile.sh`), then collect stats and plot:

```
# Usage: sudo bash run_stats_benchmark.sh [MB [stride-4k]]
# Default: 512 MB, sequential. Pass "stride-4k" for strided access.
sudo bash run_stats_benchmark.sh 4096 stride-4k # 4 GB, strided
python3 plot_backends.py # writes backend_stride.pdf, backend_gemm.pdf
```

Query-Frequency Sensitivity

```
# Tunables (override via env): SLEEP_INTERVALS, NUM_EPOCHS, REPS, benchmark filter.
# Defaults: SLEEP_INTERVALS="0.01 0.05 0.10 0.50 1.00", NUM_EPOCHS="1 5 10", REPS=5.
# Example: run only int_set_seq at 3 reps, custom epoch counts.
NUM_EPOCHS="1 3 10" sudo -E bash run_overhead_benchmark_windowed.sh 3 int_set_seq
python3 plot_query_freq.py # writes overhead_query_freq.pdf
```

Declaration

We declare that all work presented in this report is our own unless explicitly attributed to an external source. We have read and understood the Institute's policy on plagiarism and academic integrity, and confirm that this report is in compliance with those standards.